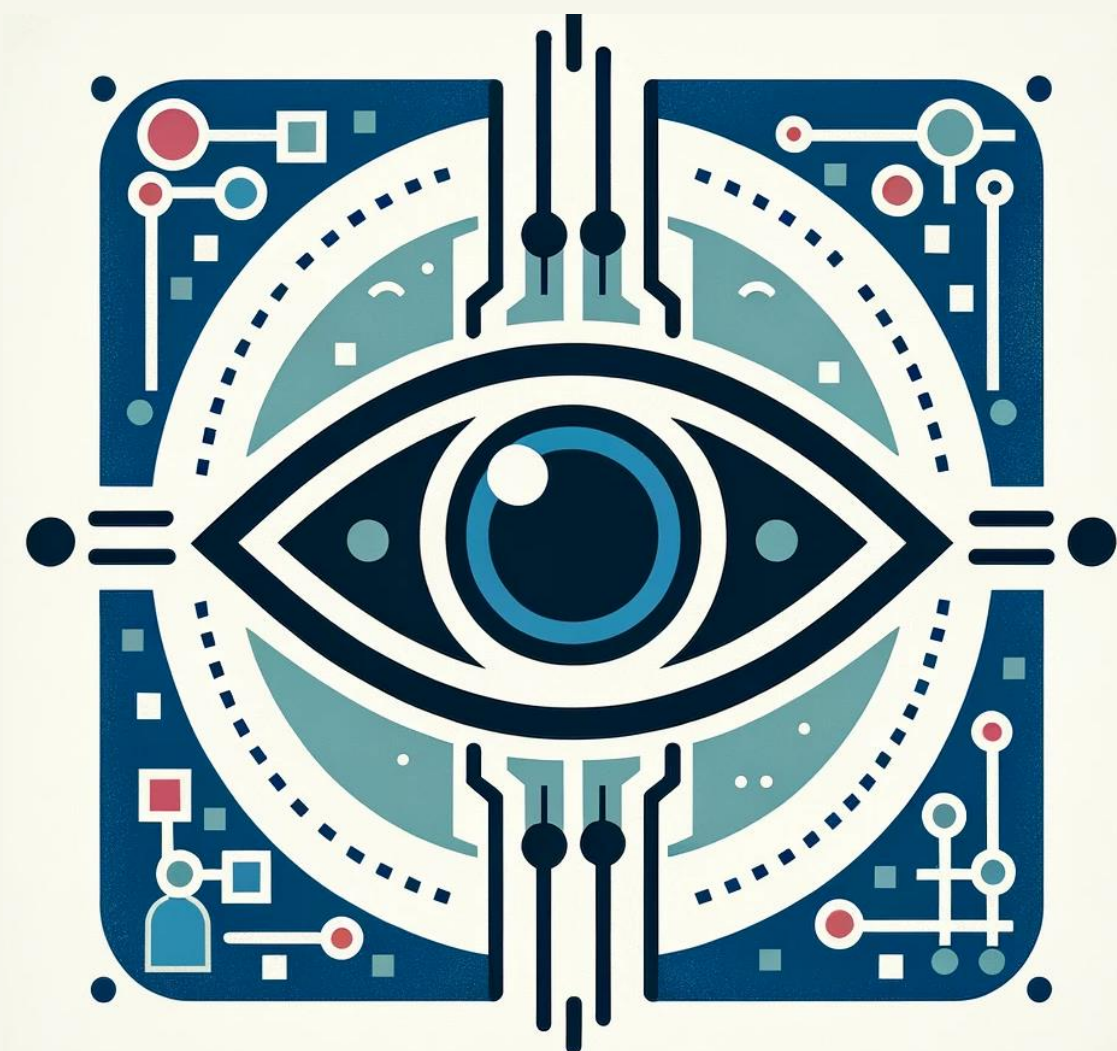




COMPUTER VISION

Deep Generative Models
VAEs, GANs, Diffusion Models

How to solve any vision problem

Before tackling, image generation...

Previously, we've covered a lot of techniques for solving vision problems with neural networks / deep learning.

Image classification

Optimize Log-likelihood / multi-class cross-entropy

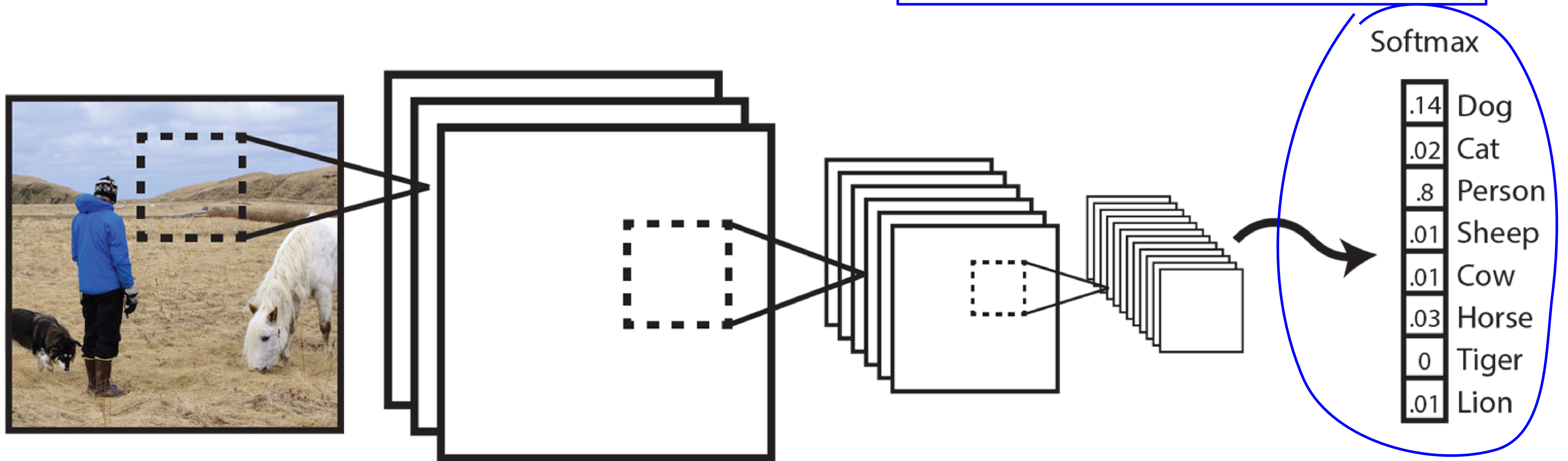
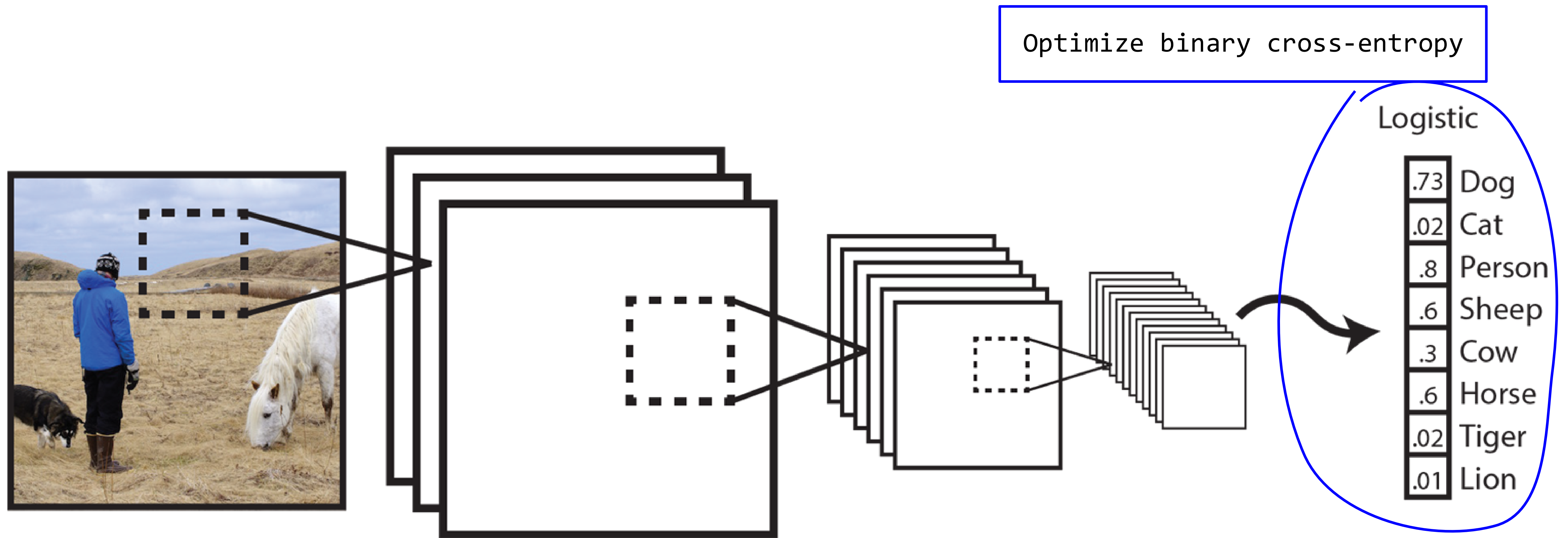
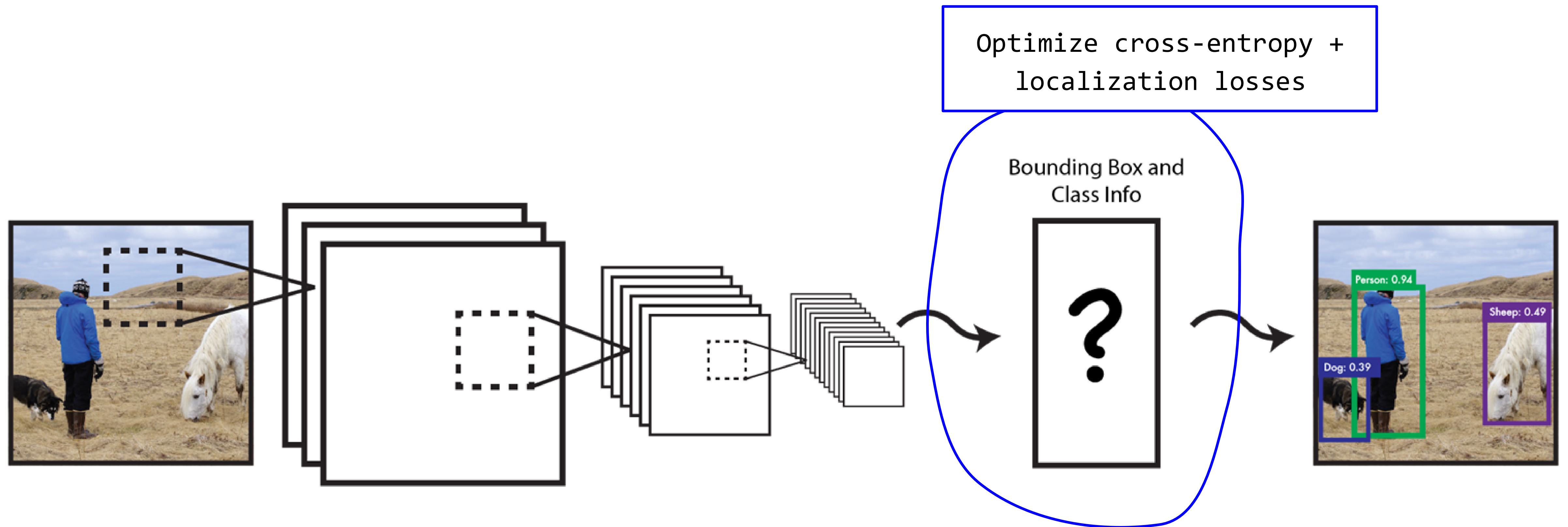


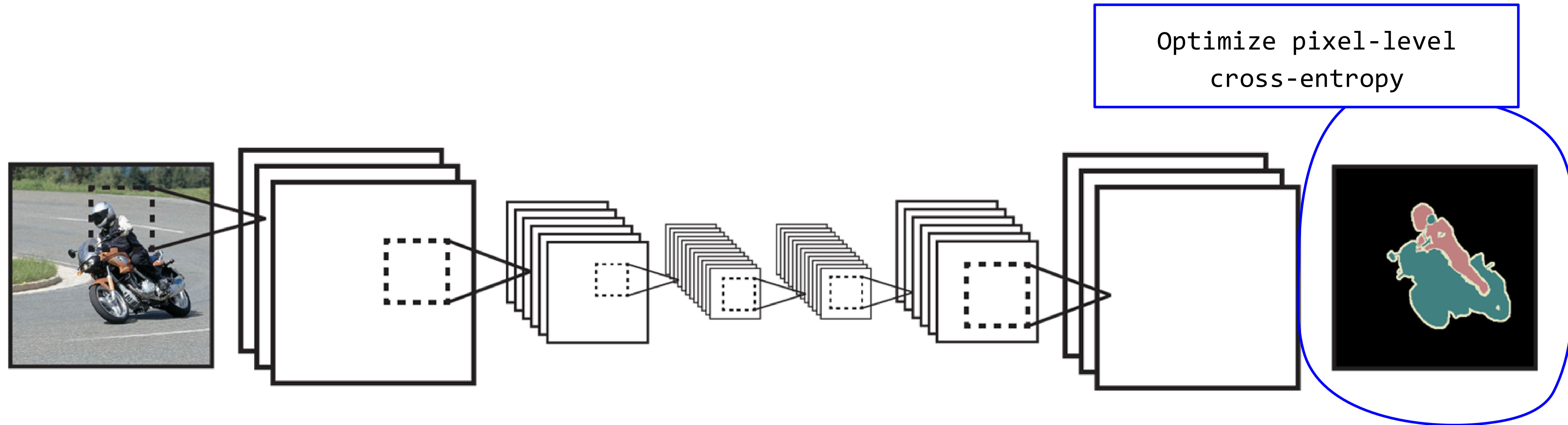
Image tagging



Object detection

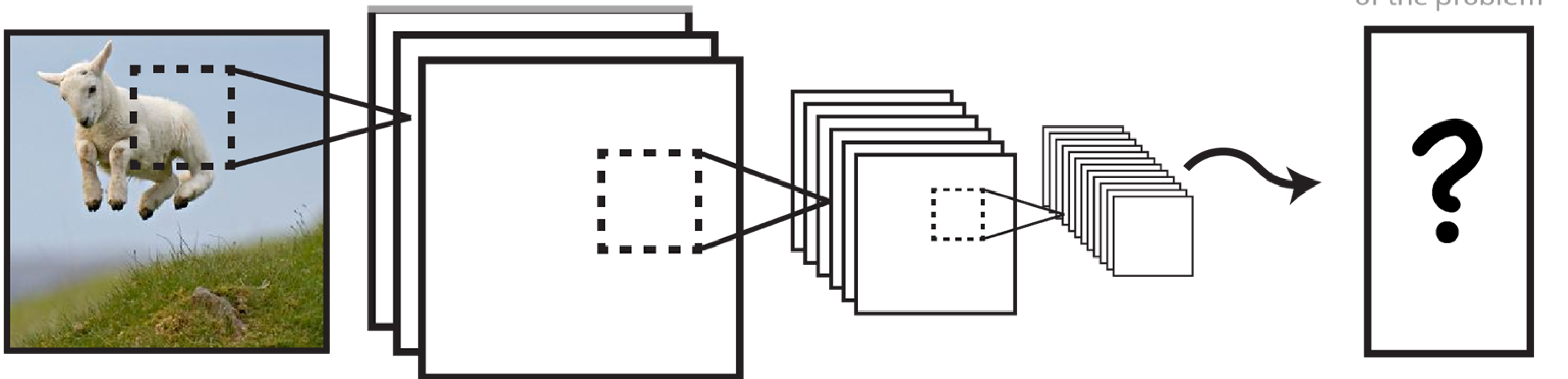


Semantic segmentation



How to solve any vision problem

1. Pick a neural network architecture
2. Design an encoding of the expected output
3. Pick a loss function for that encoding (squared error? Log-likelihood?)
4. Gather a bunch of training data (and label it)
5. Train your network with backpropagation for a long time



Can we generate images from scratch?

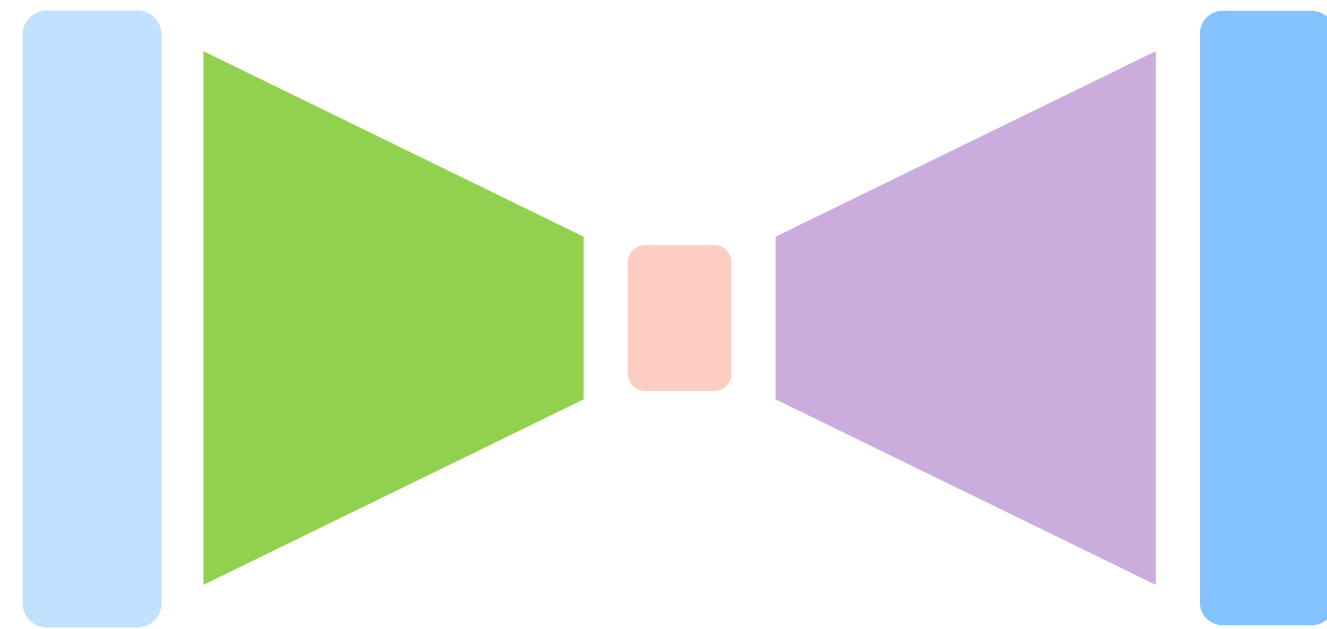


Which one is computer-generated?

Generative networks

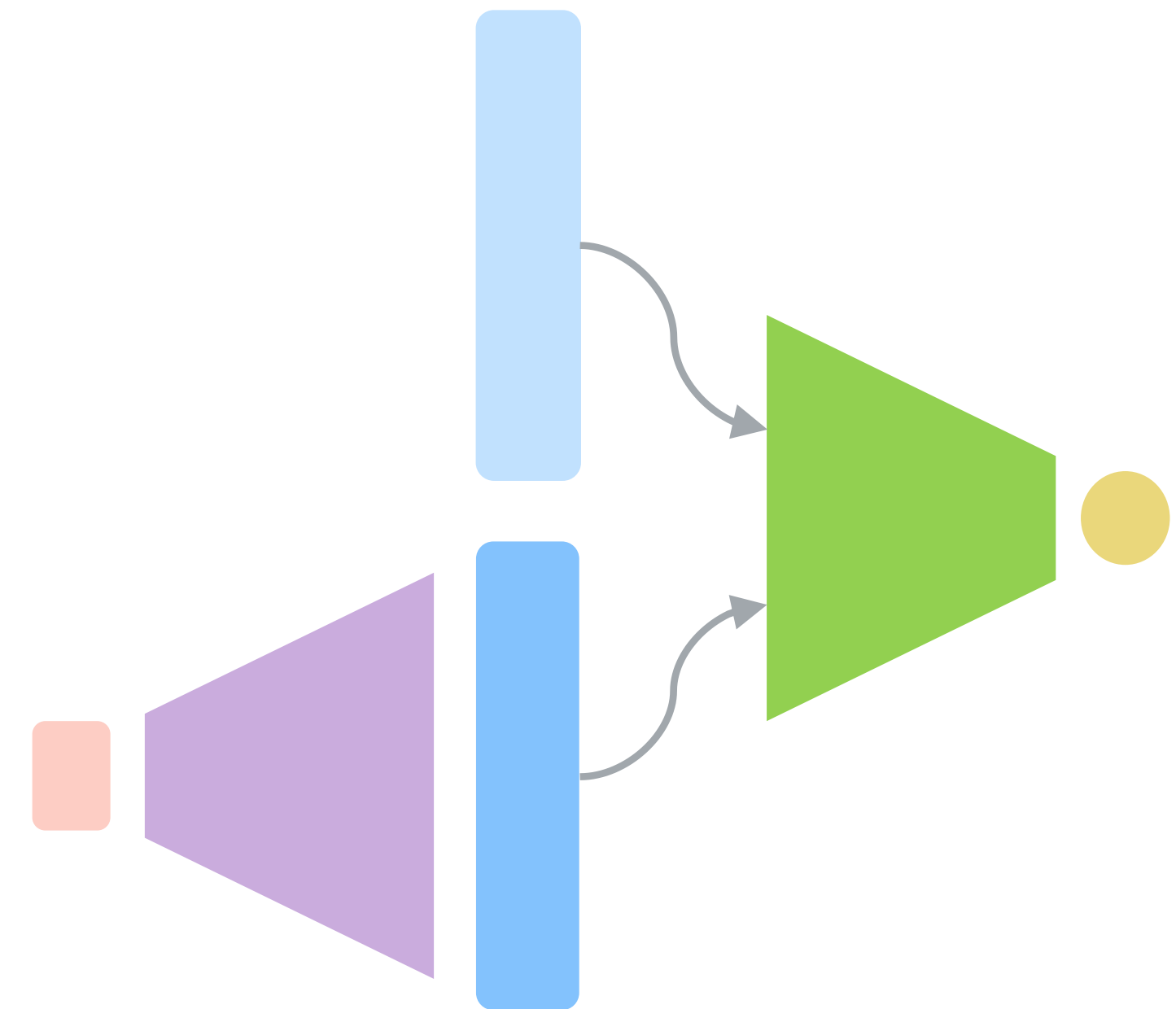
Autoencoders and Variational Autoencoders (VAEs)

Learn **lower-dimensional** latent space and **sample** to generate input reconstructions



Generative Adversarial Networks (GANs)

Competing **generator** and **discriminator** networks



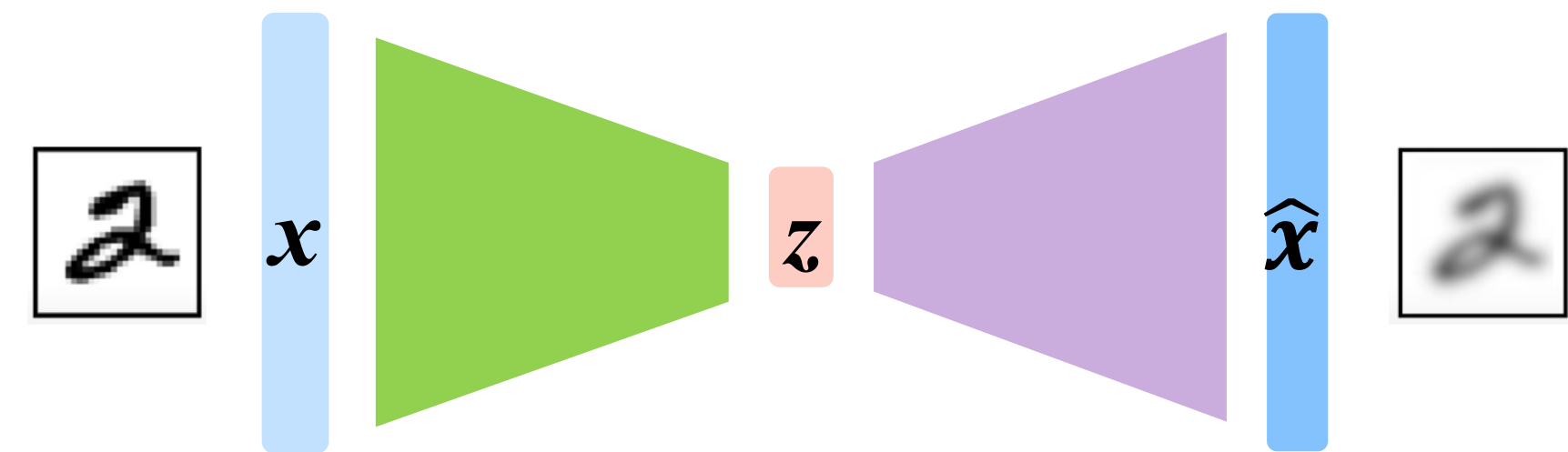
Latent variable models

- **Latent variables** are variables that we can not observe/measure directly but that are responsible for the observable variables
- We want to **learn these latent variables from the observable data**
 - NNs are well suited for this problem because of their capability of approximating complex data distributions

Autoencoders

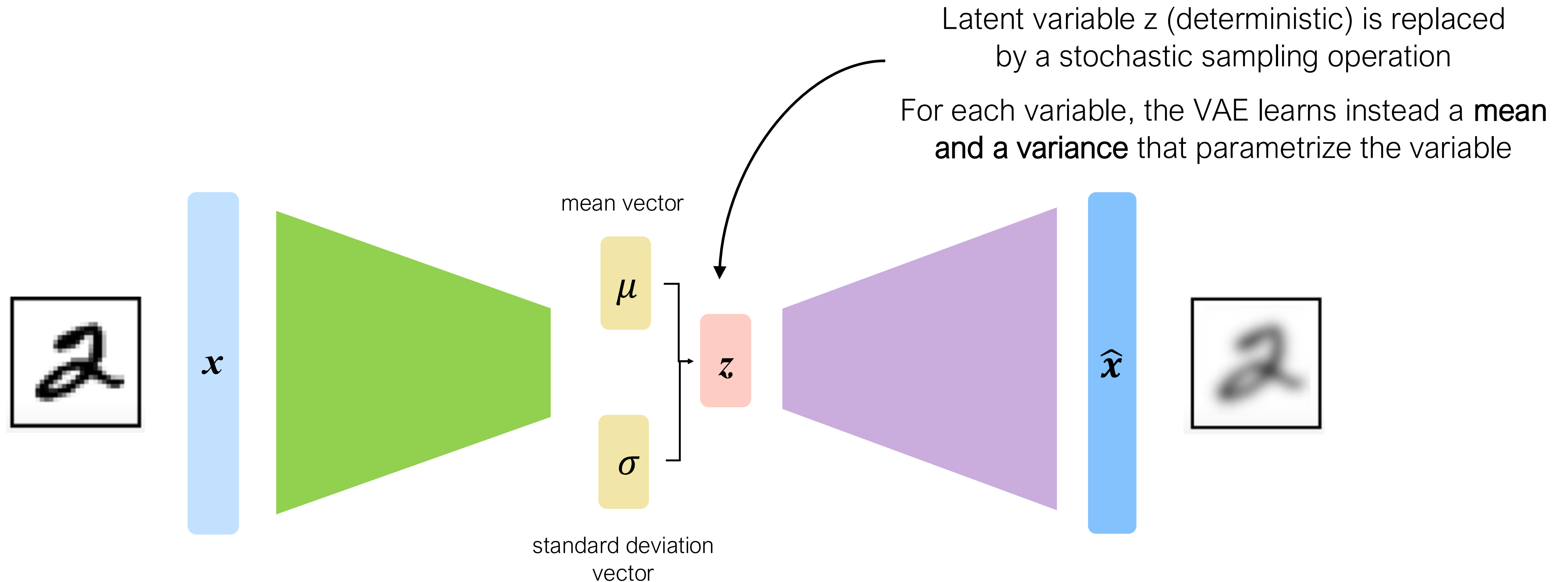
Bottleneck hidden layer forces the network to learn a compressed latent representation

Reconstruction loss forces the latent space to capture (or encode) as much “information” about the data as possible?



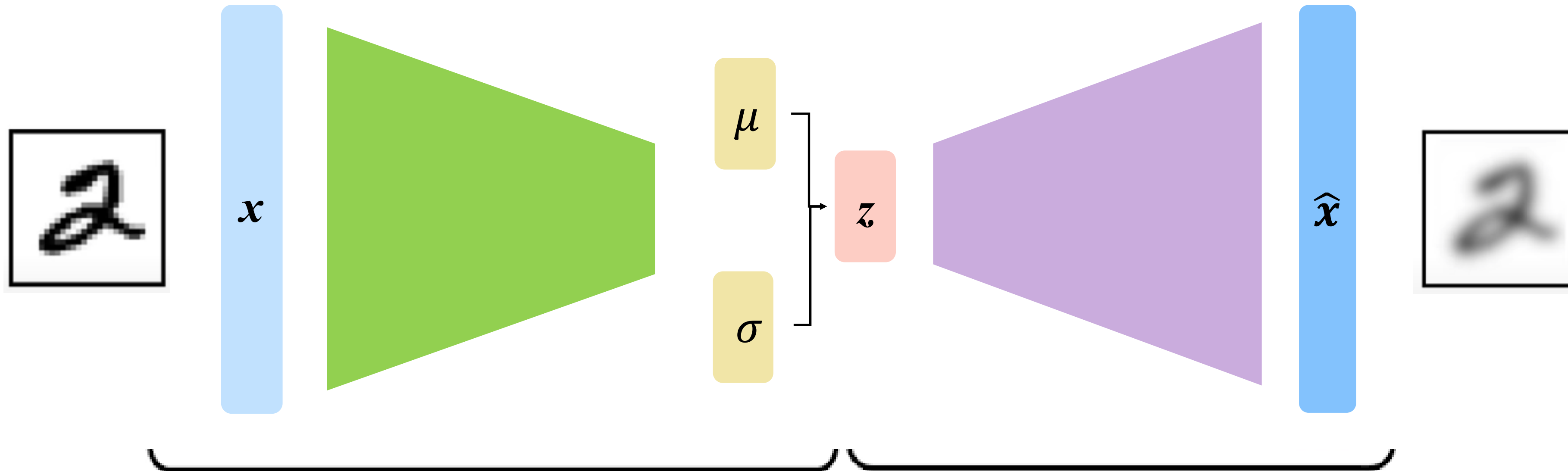
The learned representation is deterministic.
Is that enough for image generation?

Variational Autoencoders (VAEs)



After training, use the generator to create new examples

VAE optimization



Encoder computes: $q_{\phi}(z|x)$

Decoder computes: $p_{\theta}(x|z)$

$\mathcal{L}(\phi, \theta, x) = (\text{reconstruction loss}) + (\text{regularization term})$

$$\downarrow$$
$$\|x - \hat{x}\|^2$$

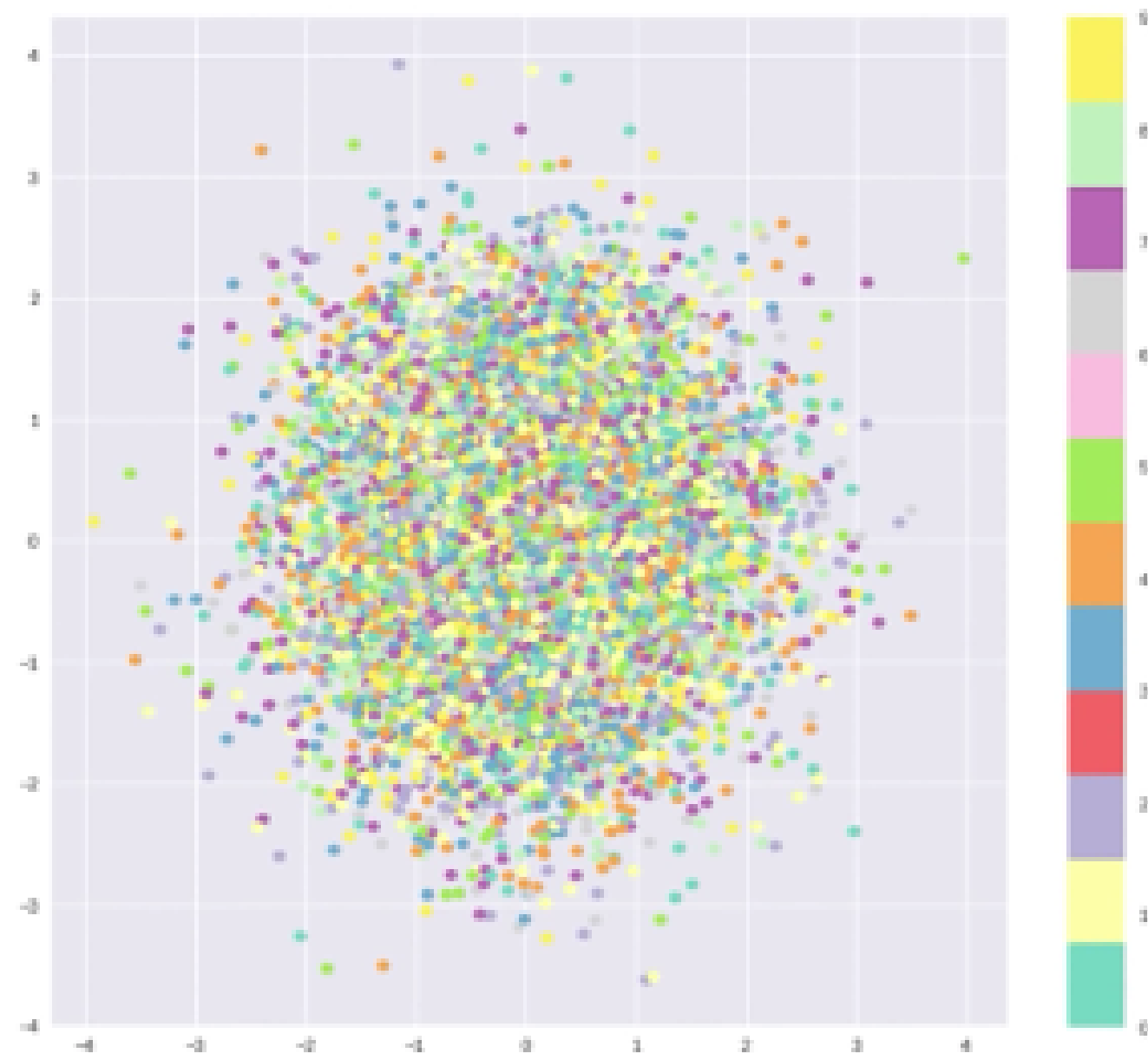
$$\downarrow$$
$$D(q_{\phi}(z|x) || p(z))$$

VAE optimization

$$D \left(q_{\phi}(z|x) \parallel p(z) \right)$$

Inferred latent distribution Fixed prior on latent distribution

minimize the **divergence** between the **learned latent distribution** and a **specified prior** (initial hypothesis about the distribution of z) → encourage the network to learn a distribution similar to the prior



Common choice of prior – Normal Gaussian:

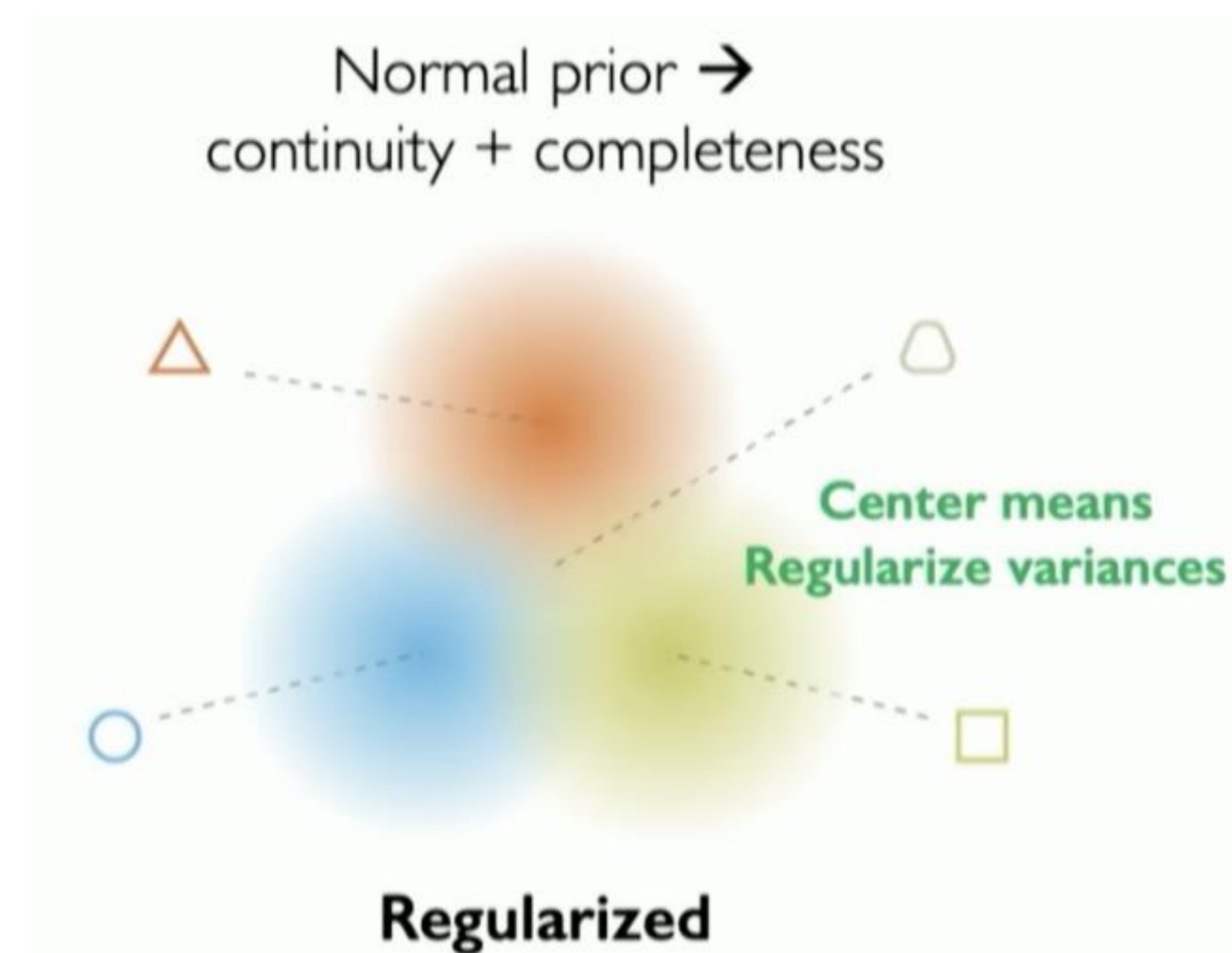
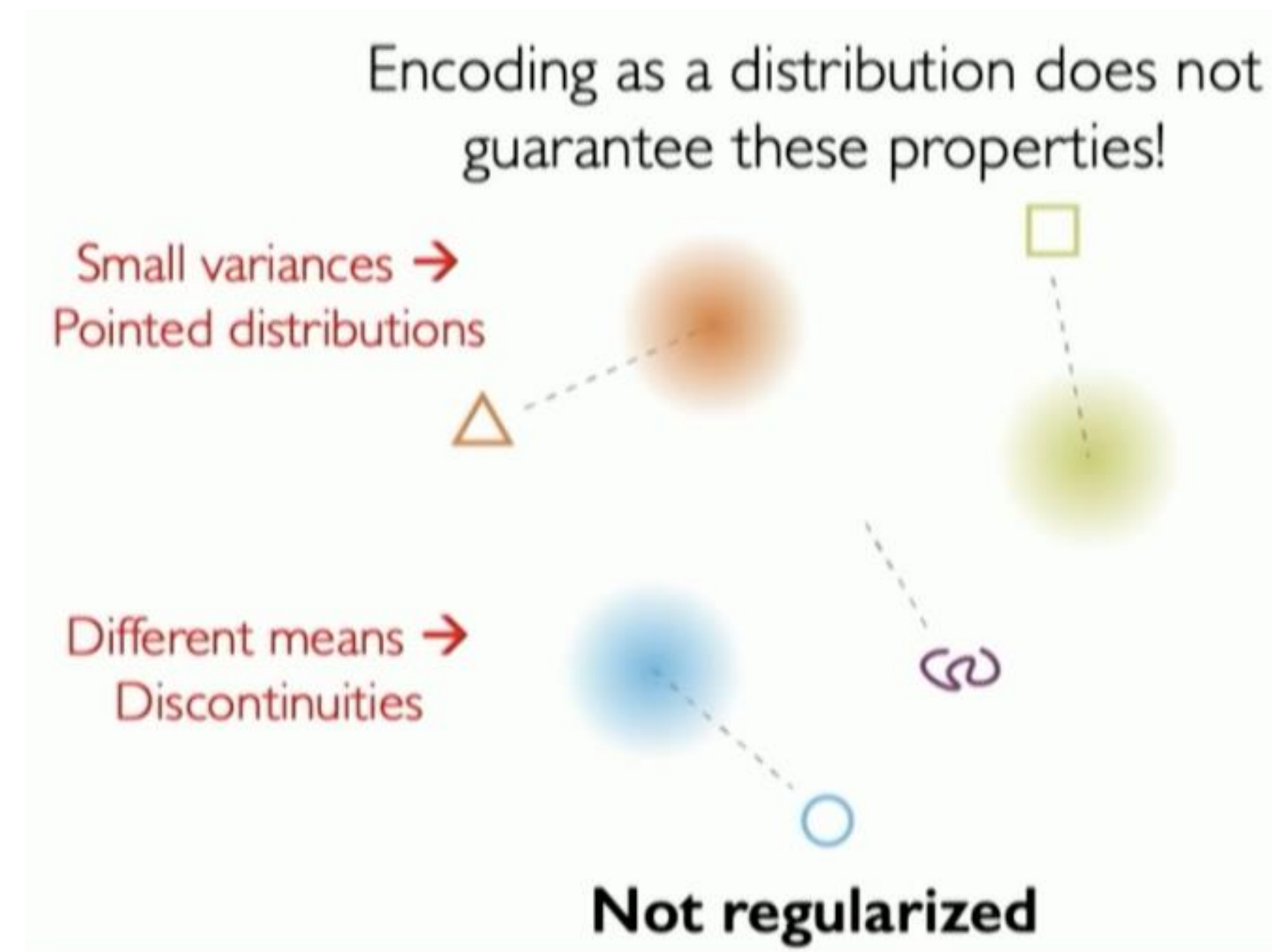
$$p(z) = \mathcal{N}(\mu = 0, \sigma^2 = 1)$$

- Encourages encodings to distribute evenly around the center of the latent space
- Penalize the network when it tries to “cheat” by clustering points in specific regions (i.e., by memorizing the data)

Regularization + Normal Prior

What properties do we want to achieve from regularization?

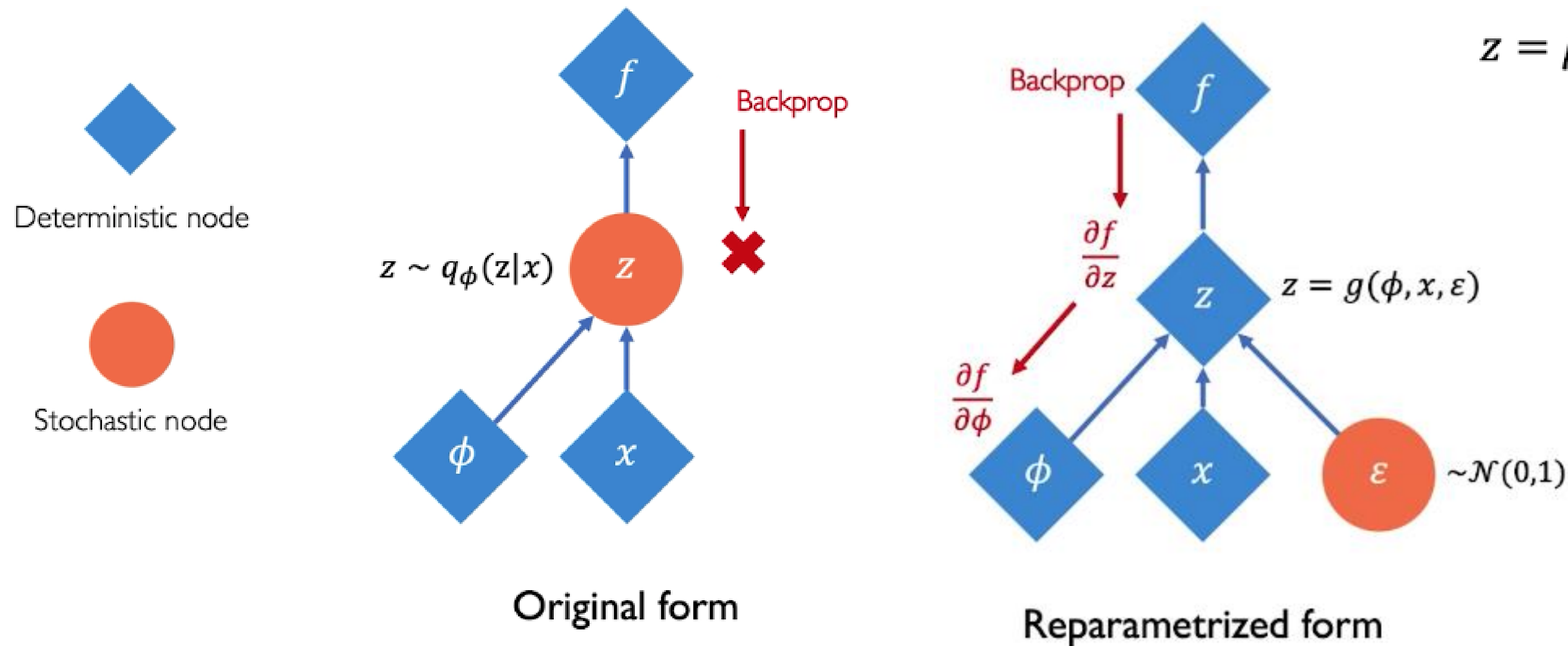
- **Continuity**: points are close in latent space $\rightarrow$ similar content after decoding
- **Completeness**: sampling from latent space $\rightarrow$ “meaningful” content after decoding



Trade-off: the more we regularize, the higher the risk the quality of the reconstruction will suffer

VAE optimization

Sampling layer is **not differentiable** → solution: reparametrize the sampling layer



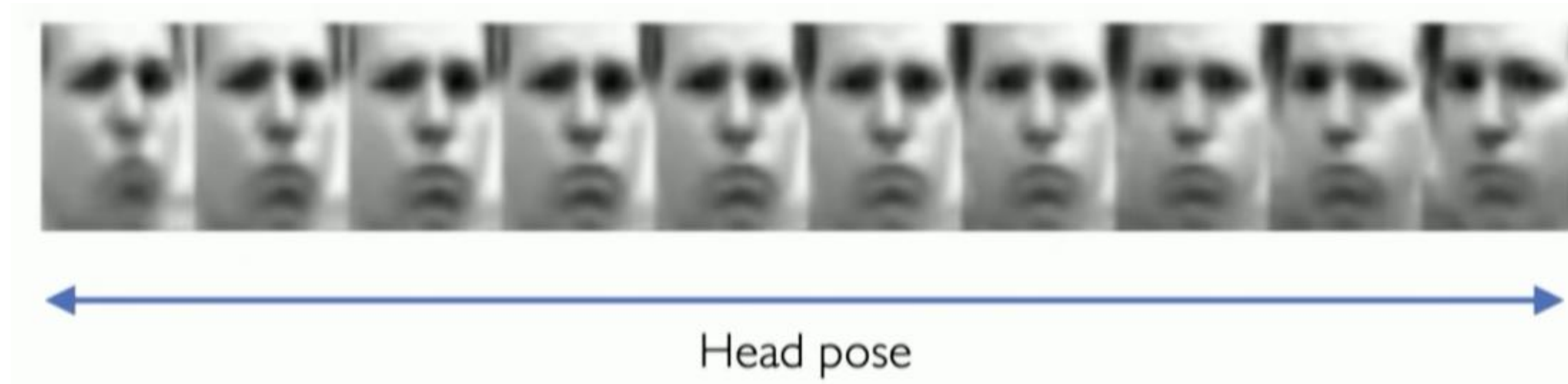
$$z = \mu + \sigma \odot \varepsilon$$

where $\varepsilon \sim \mathcal{N}(0,1)$

Stochasticity is moved to ε (an external input), while μ and σ become **deterministic** functions of the encoder.

VAE latent variables

Slowly increase or decrease a **single latent variable**
Keep all other variables fixed



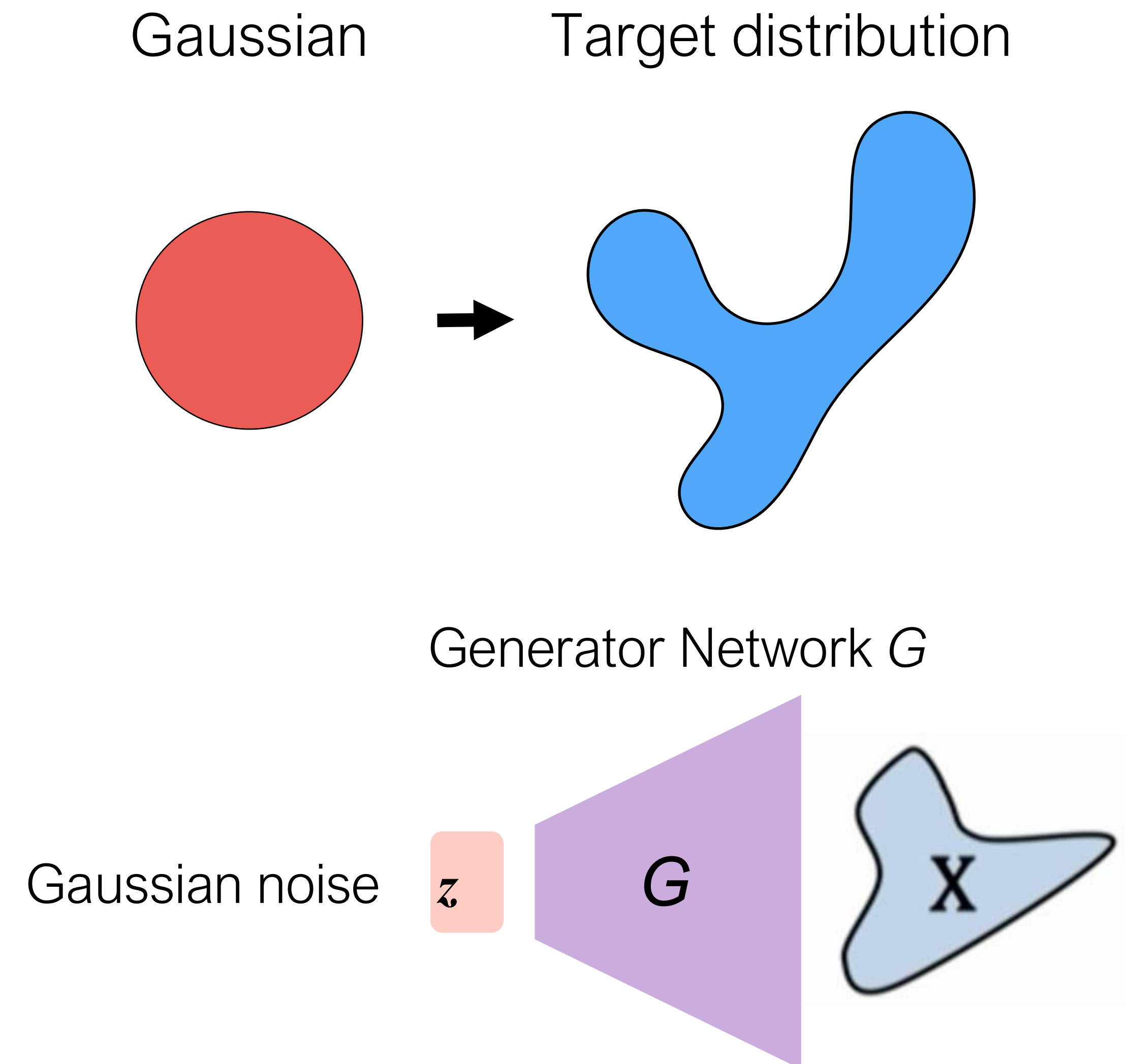
Different dimensions of z encodes **different interpretable latent features**

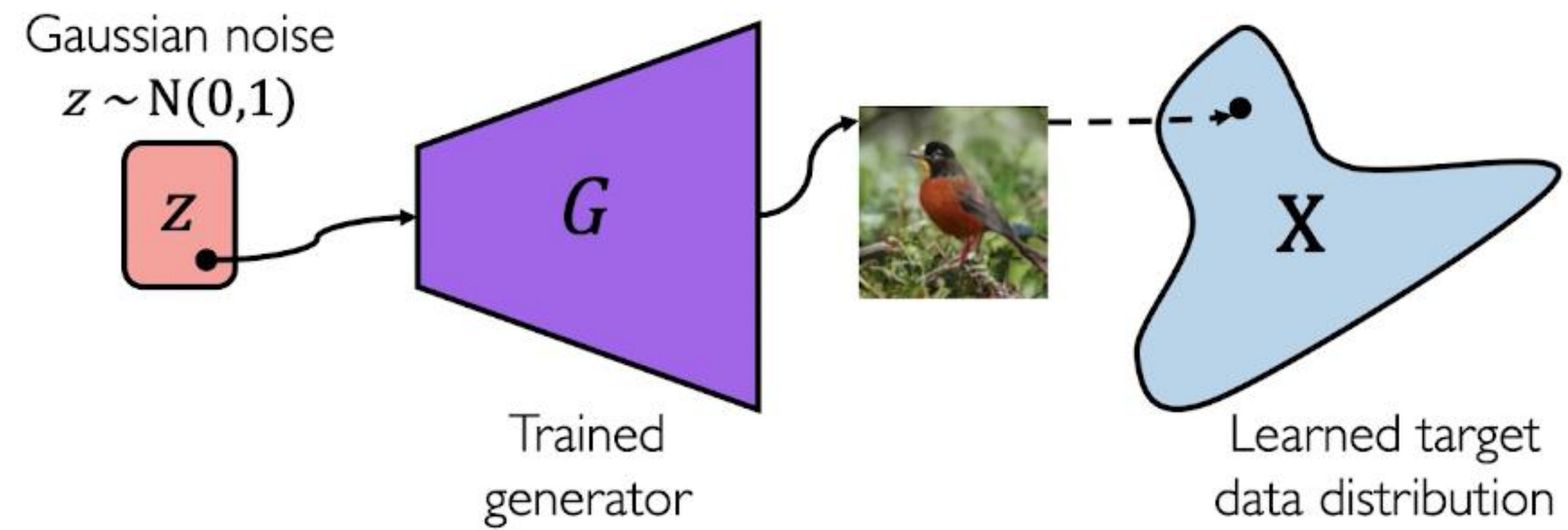
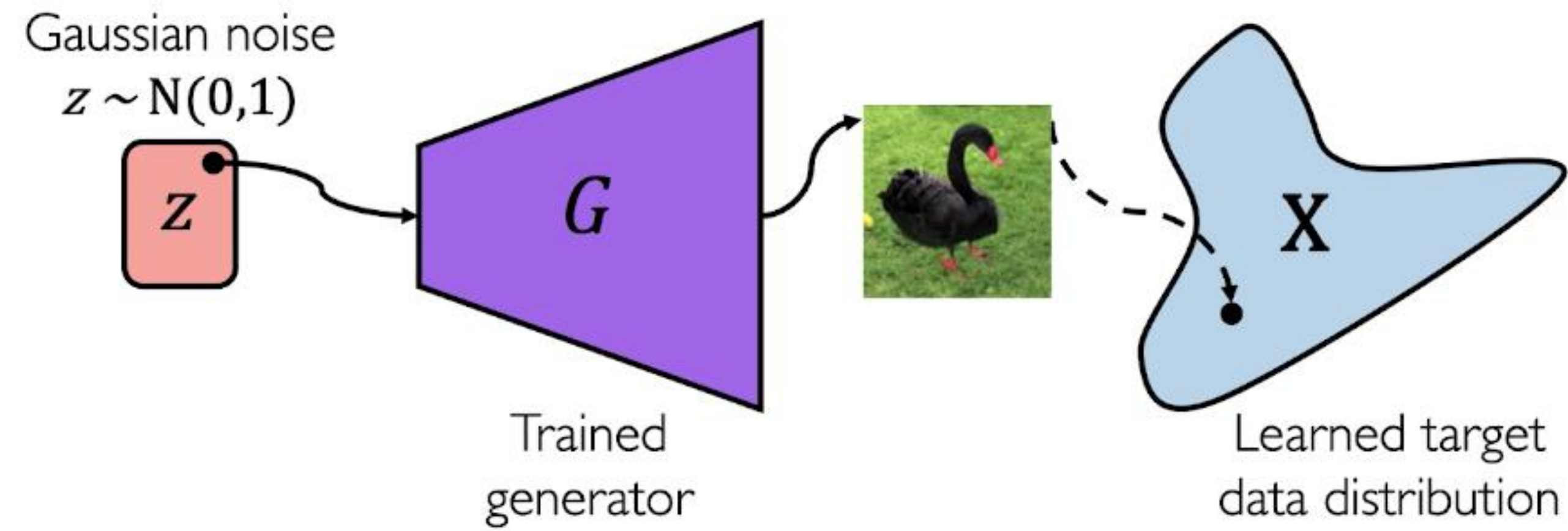
What if we just want to sample?

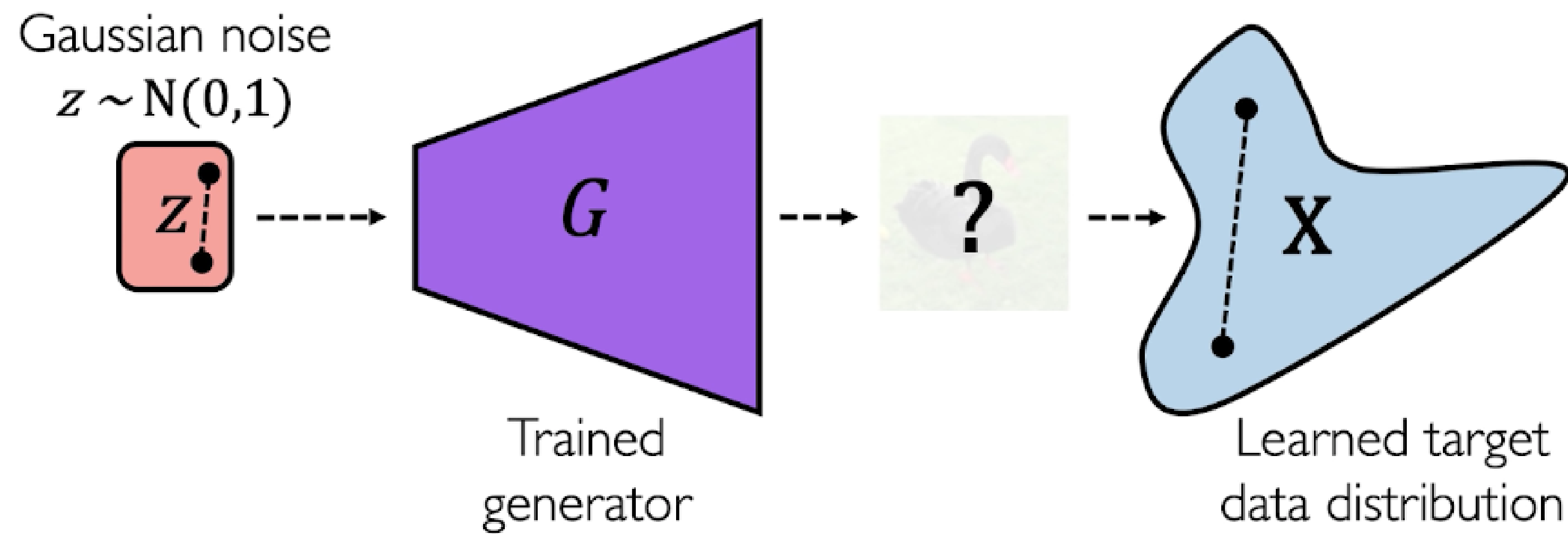
Idea: don't explicitly model density, and instead just sample to generate new instances

Problem: want to sample from complex distributions – can't do this directly

Solution: sample from something simple (noise), learn a transformation to the training distribution







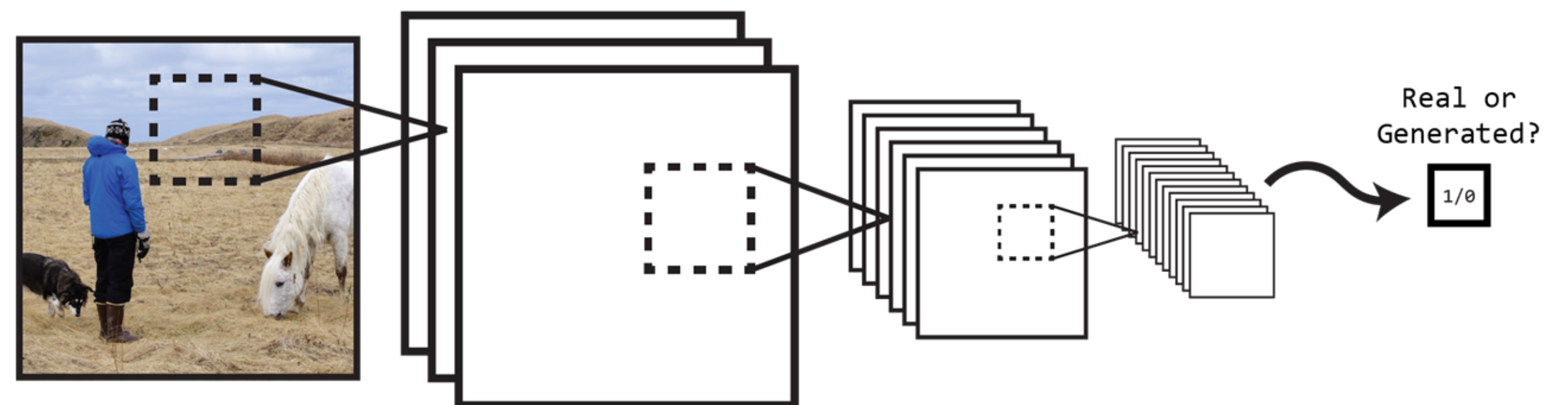
How can we train the generator?

What loss function can we use?

What we really want is... does the image look good?

→ We need some kind of perceptual loss

How? Well, we just spent a long time learning how to train neural networks to do stuff...

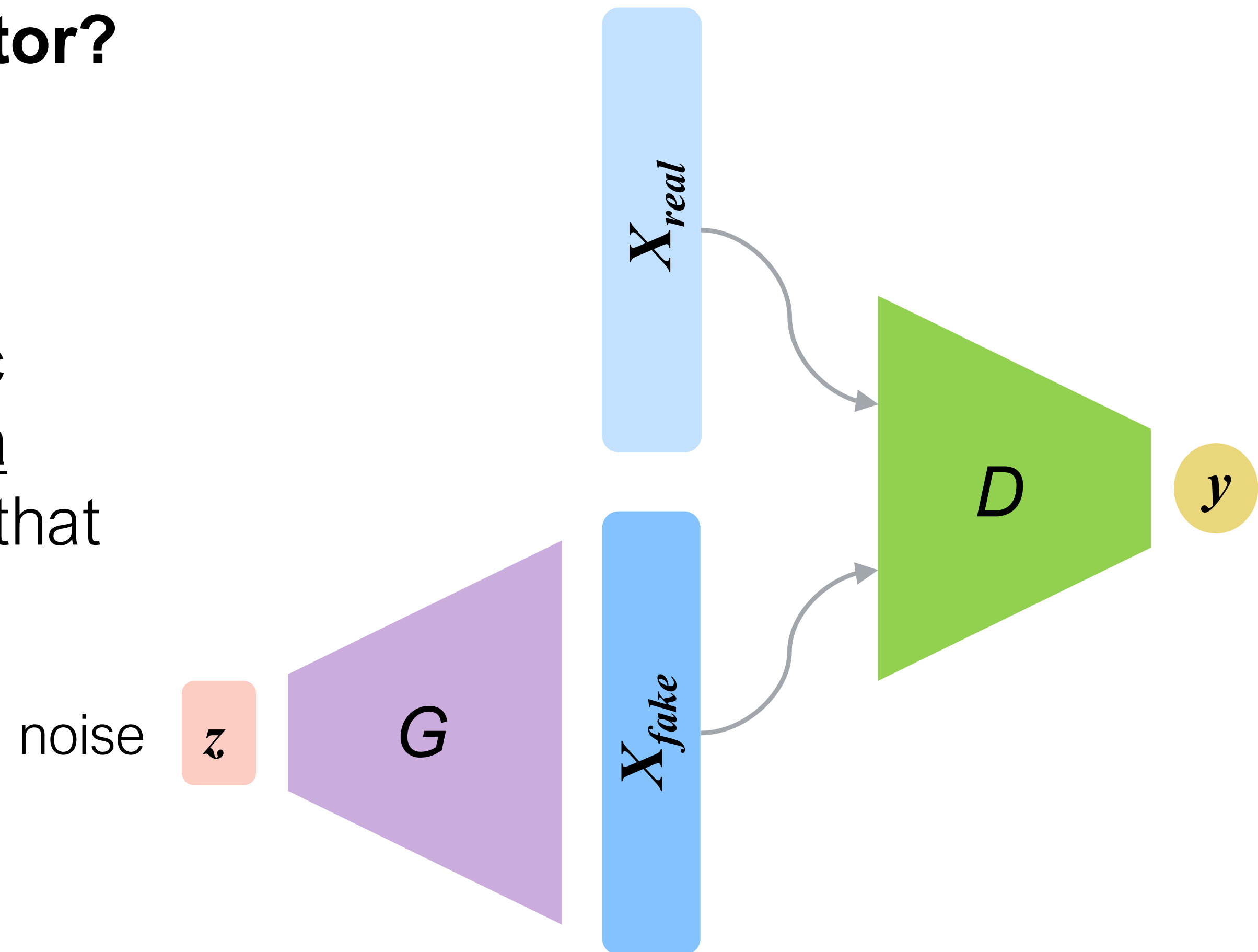


Generative Adversarial Networks (GANs)

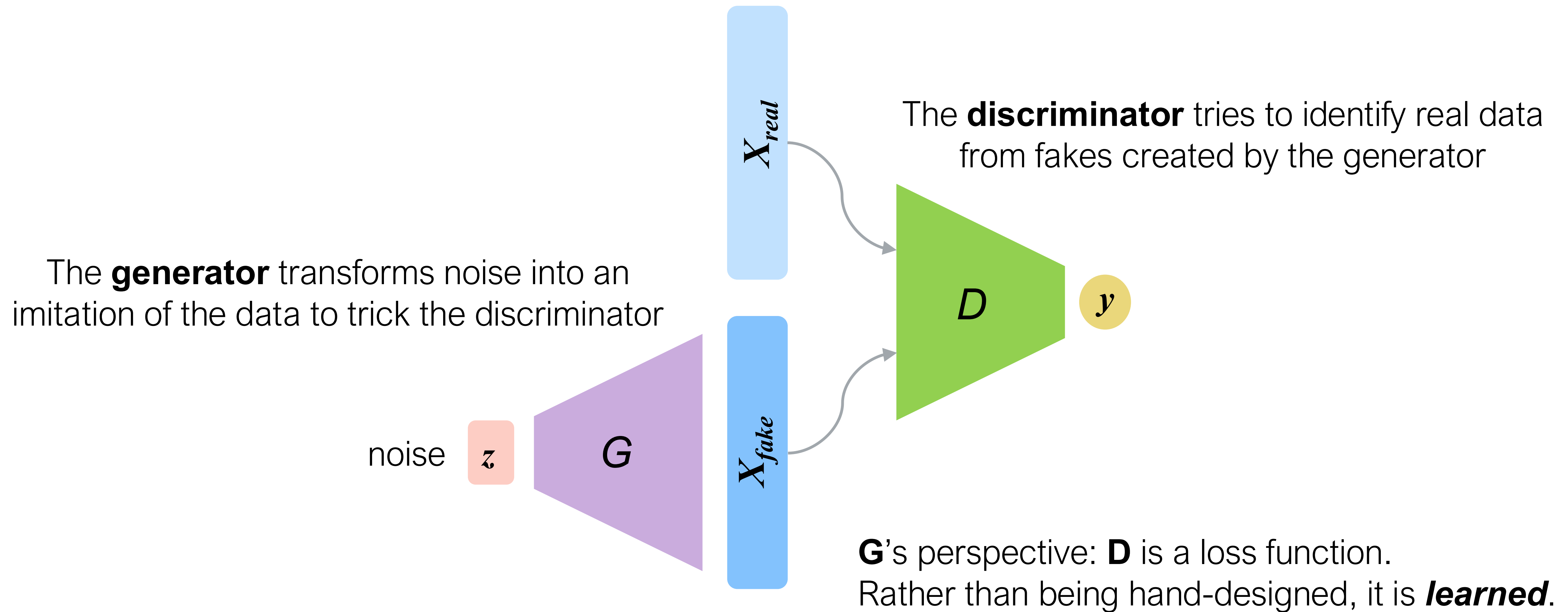
How can we train the generator?

Adversarial Training

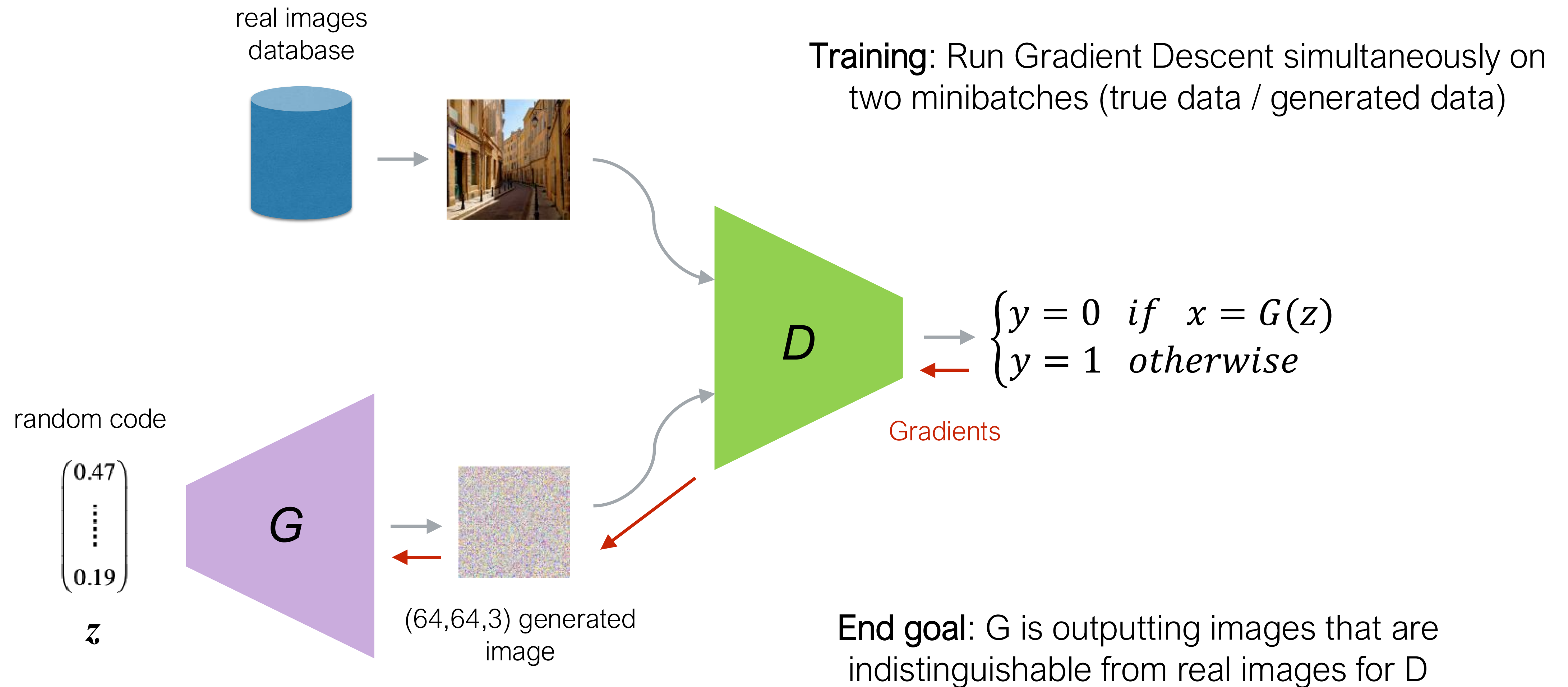
the generator is not trained to minimize a distance to a specific input or distribution, but to fool a discriminator (binary classifier), that at the same time learns to discriminate better



Generative Adversarial Networks (GANs)



Generative Adversarial Networks (GANs)



Generative Adversarial Networks (GANs)

Training procedure, we want to minimize:

The cost of the **discriminator**

Labels: $\begin{cases} y_{real} & \text{is always 1} \\ y_{gen} & \text{is always 0} \end{cases}$

$$\mathcal{L}_D = \underbrace{-\frac{1}{m_{real}} \sum_{i=1}^{m_{real}} y_{real}^{(i)} \cdot \log(D(x^{(i)}))}_{\text{cross-entropy 1: "D should correctly label real data as 1"}} - \underbrace{\frac{1}{m_{gen}} \sum_{i=1}^{m_{gen}} (1 - y_{gen}^{(i)}) \cdot \log(1 - D(G(z^{(i)})))}_{\text{cross-entropy 2: "D should correctly label generated data as 0"}}$$

The cost of the **generator**

$$\mathcal{L}_G = -\mathcal{L}_D = \frac{1}{m_{gen}} \sum_{i=1}^{m_{gen}} \log(1 - D(G(z^{(i)})))$$

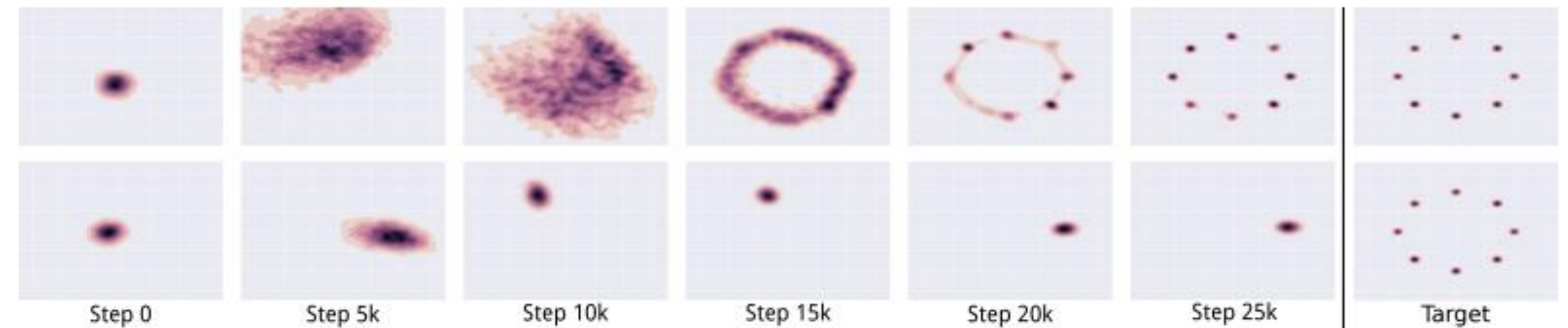
“G should try to fool D: by minimizing the opposite of what D is trying to minimize”

GANs Difficulties

Mode collapse

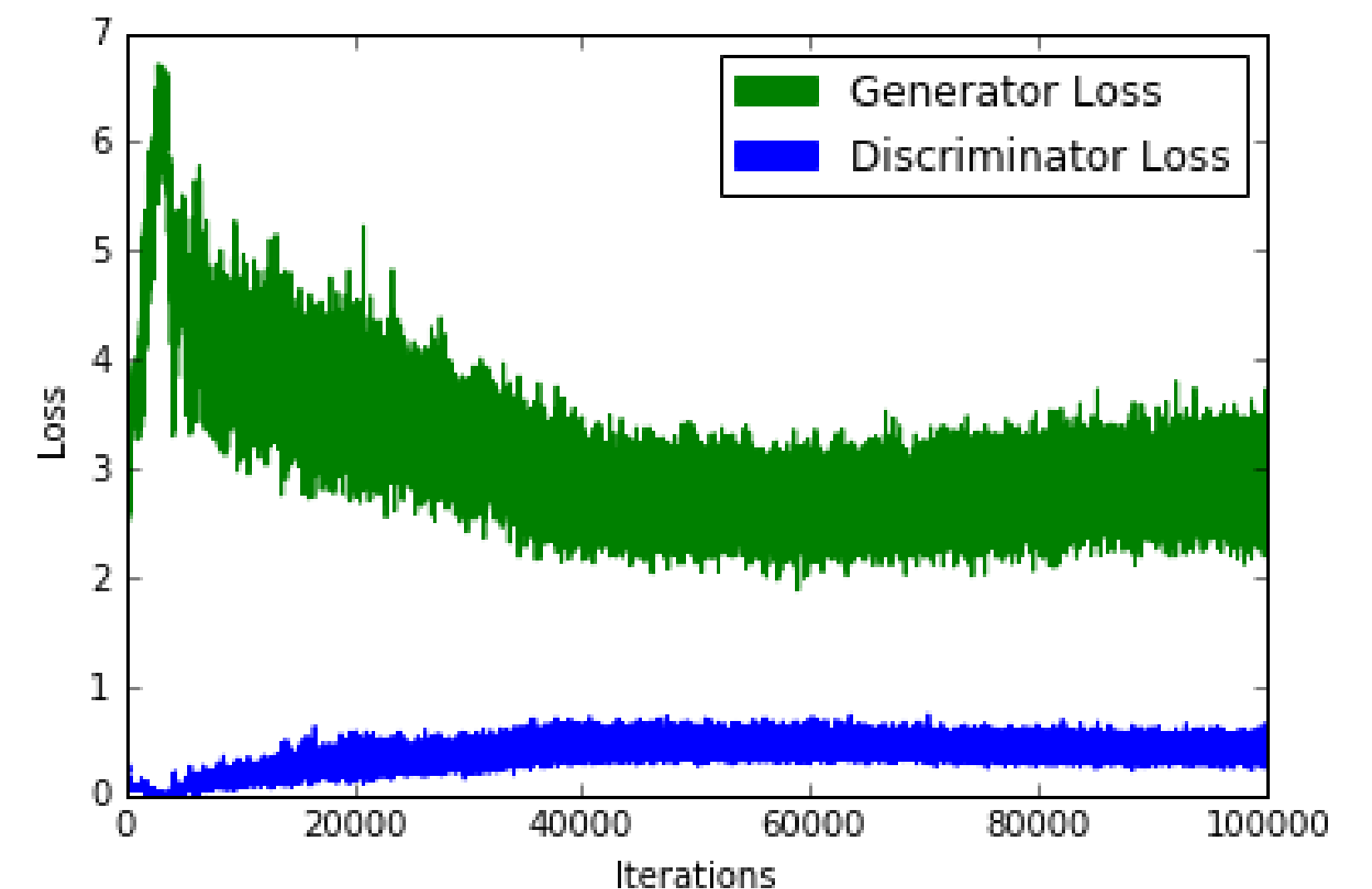
- Natural data distributions are highly complex and multimodal (many modes)
- Each **mode** represents a concentration of **similar data samples**
- During mode collapse, the generator produces samples that belong to a **limited set of modes**

mode collapse →



Convergence

- When to stop the training process?
- Since the generator loss improves when the discriminator loss degrades (and vice-versa)



GANs Difficulties

Quality metric

- How to quantitatively evaluate the quality of the generated samples?

Fréchet Inception Distance (FID)

- Uses Fréchet Distance to compare the **feature maps** produced by passing the real and generated images through a pre-trained Inception-v3 Network
- A **lower FID score is better**, as it explains that the statistics of the generated images are very similar to that of the real images.

Evaluation Beyond FID

Known FID limitations

Sensitive to sample size; biased toward Inception's feature space; doesn't capture all failure modes (e.g., texture artifacts, semantic errors).

CLIP Score

Cosine similarity between CLIP embeddings of generated image and text prompt → measures text-image alignment for conditional generation.

Precision & Recall for generative models

Disentangles fidelity (precision: are samples realistic?) from diversity (recall: does the model cover the data distribution?).

GANs Difficulties

- Training is really hard
 - Unstable
 - Classification easier than generation

Stabilizing GAN Training

Why GANs are unstable

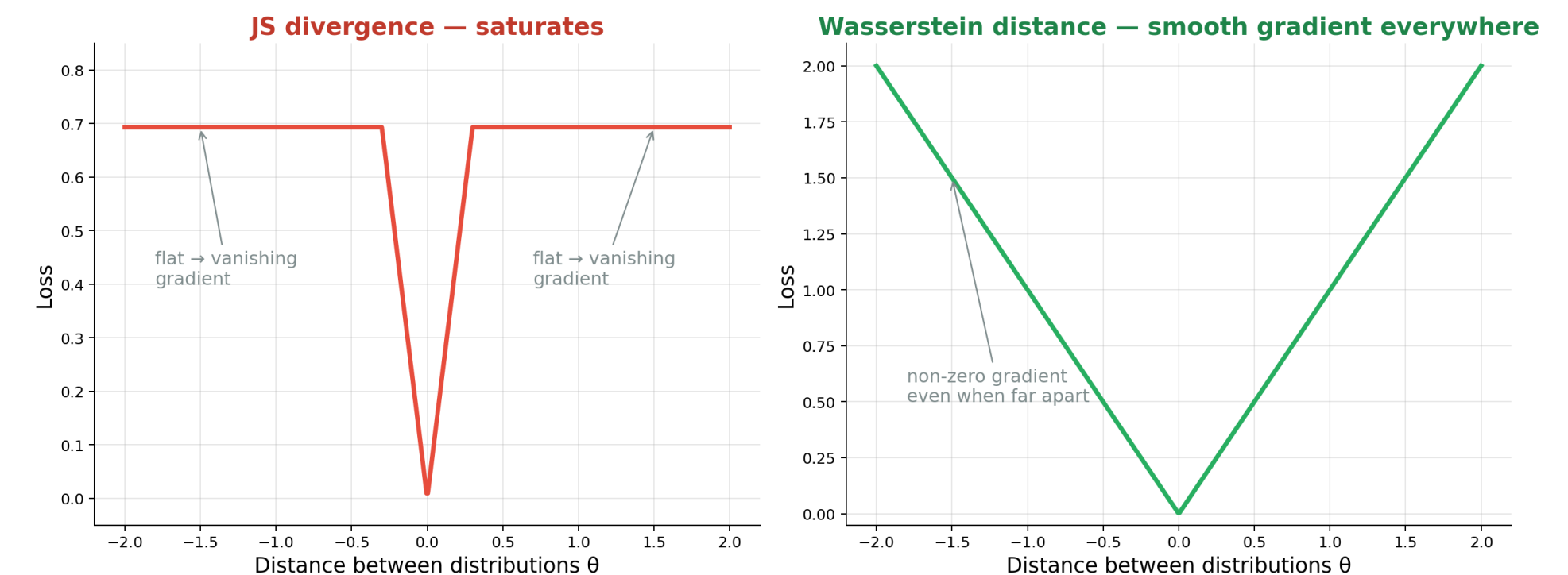
Standard GAN minimizes Jensen-Shannon divergence → **vanishing gradients** when real and generated distributions don't overlap.

Wasserstein GAN (WGAN)

Replaces JS divergence with the Wasserstein (Earth Mover's) distance → **smoother, non-vanishing gradients** even when distributions are far apart.

Discriminator becomes a critic that outputs a real-valued score, not a probability.

Requires Lipschitz constraint: weight clipping (WGAN) or gradient penalty (WGAN-GP).



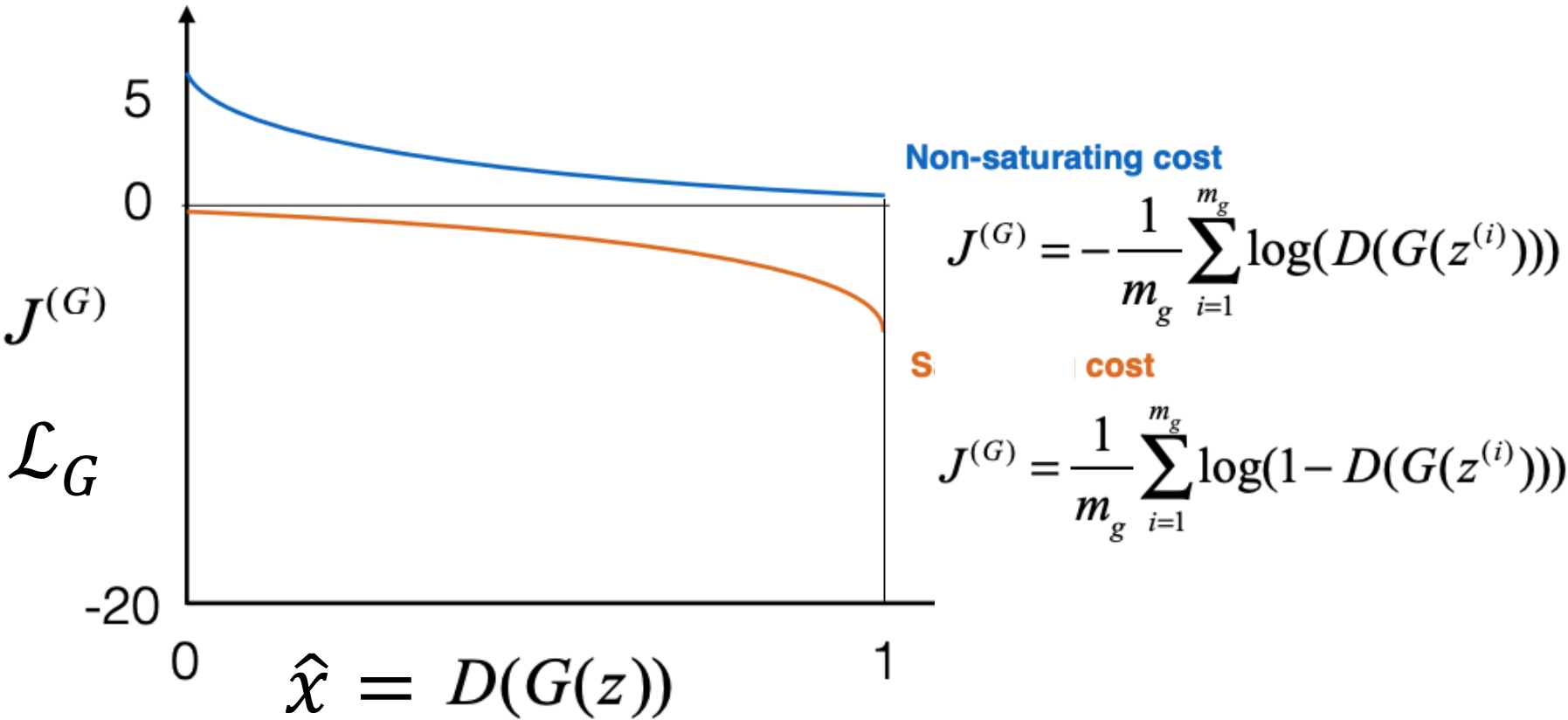
Other stabilization techniques

Spectral normalization, two time-scale update rule (TTUR), label smoothing, instance noise.

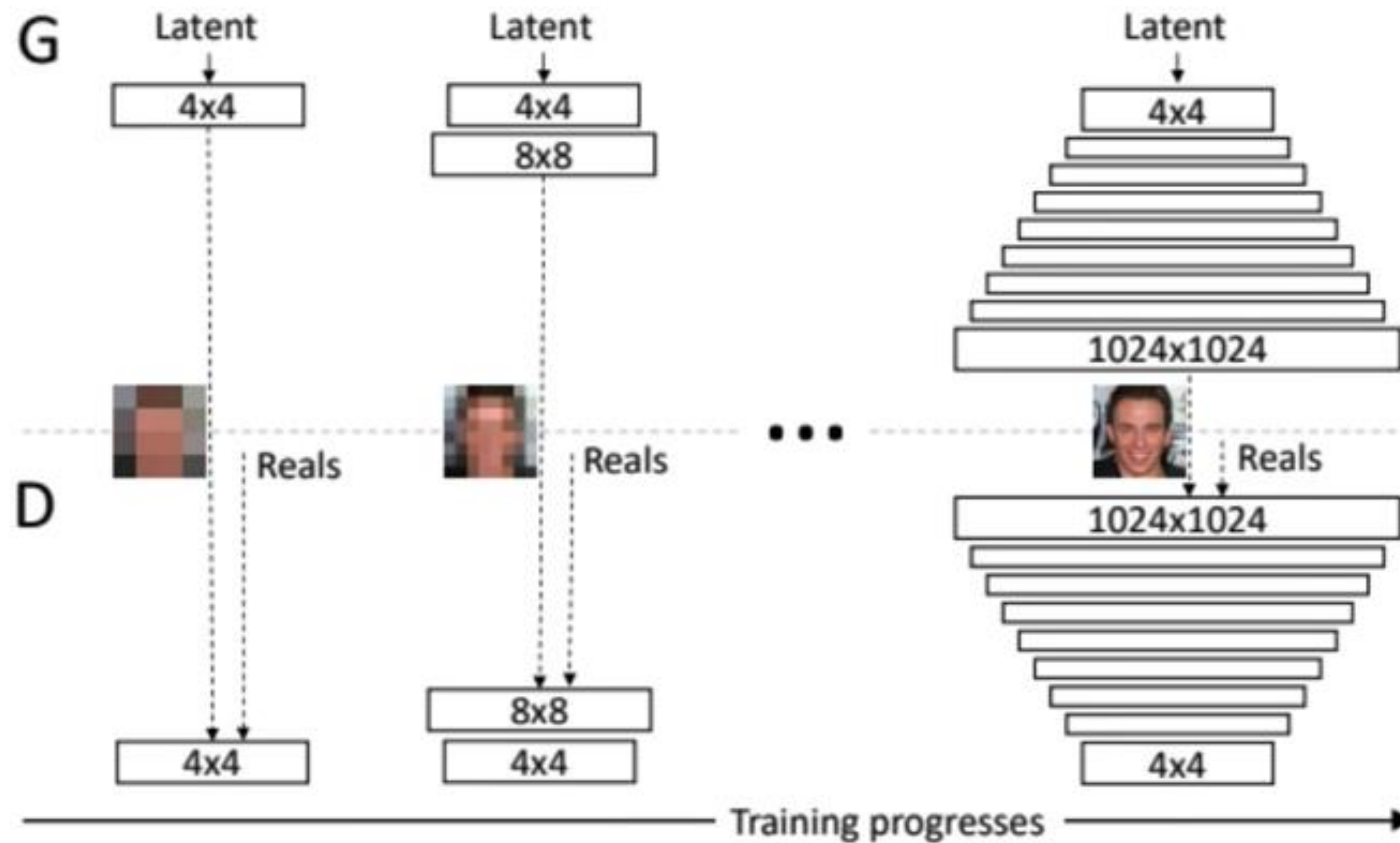
GAN	DISCRIMINATOR LOSS	GENERATOR LOSS
MM GAN	$\mathcal{L}_D^{\text{GAN}} = -\mathbb{E}_{x \sim p_d} [\log(D(x))] - \mathbb{E}_{\hat{x} \sim p_g} [\log(1 - D(\hat{x}))]$	$\mathcal{L}_G^{\text{GAN}} = \mathbb{E}_{\hat{x} \sim p_g} [\log(1 - D(\hat{x}))]$
NS GAN	$\mathcal{L}_D^{\text{NSGAN}} = -\mathbb{E}_{x \sim p_d} [\log(D(x))] - \mathbb{E}_{\hat{x} \sim p_g} [\log(1 - D(\hat{x}))]$	$\mathcal{L}_G^{\text{NSGAN}} = -\mathbb{E}_{\hat{x} \sim p_g} [\log(D(\hat{x}))]$
WGAN	$\mathcal{L}_D^{\text{WGAN}} = -\mathbb{E}_{x \sim p_d} [D(x)] + \mathbb{E}_{\hat{x} \sim p_g} [D(\hat{x})]$	$\mathcal{L}_G^{\text{WGAN}} = -\mathbb{E}_{\hat{x} \sim p_g} [D(\hat{x})]$
WGAN GP	$\mathcal{L}_D^{\text{WGANGP}} = \mathcal{L}_D^{\text{WGAN}} + \lambda \mathbb{E}_{\hat{x} \sim p_g} [(\ \nabla D(\alpha x + (1 - \alpha \hat{x}))\ _2 - 1)^2]$	$\mathcal{L}_G^{\text{WGANGP}} = -\mathbb{E}_{\hat{x} \sim p_g} [D(\hat{x})]$
LS GAN	$\mathcal{L}_D^{\text{LSGAN}} = -\mathbb{E}_{x \sim p_d} [(D(x) - 1)^2] + \mathbb{E}_{\hat{x} \sim p_g} [D(\hat{x})^2]$	$\mathcal{L}_G^{\text{LSGAN}} = -\mathbb{E}_{\hat{x} \sim p_g} [(D(\hat{x}) - 1)^2]$
DRAGAN	$\mathcal{L}_D^{\text{DRAGAN}} = \mathcal{L}_D^{\text{GAN}} + \lambda \mathbb{E}_{\hat{x} \sim p_d + \mathcal{N}(0, c)} [(\ \nabla D(\hat{x})\ _2 - 1)^2]$	$\mathcal{L}_G^{\text{DRAGAN}} = \mathbb{E}_{\hat{x} \sim p_g} [\log(1 - D(\hat{x}))]$
BEGAN	$\mathcal{L}_D^{\text{BEGAN}} = \mathbb{E}_{x \sim p_d} [\ x - \text{AE}(x)\ _1] - k_t \mathbb{E}_{\hat{x} \sim p_g} [\ \hat{x} - \text{AE}(\hat{x})\ _1]$	$\mathcal{L}_G^{\text{BEGAN}} = \mathbb{E}_{\hat{x} \sim p_g} [\ \hat{x} - \text{AE}(\hat{x})\ _1]$

maximize the probability of images being **real**, instead of **minimizing** the probability of an image being **fake**

Many variations of the D/G losses have been proposed in the recent years



Progressive growing of GANs





<https://www.youtube.com/watch?v=kSLJriaOumA>

<https://www.youtube.com/watch?v=c-NJtV9Jvp0>

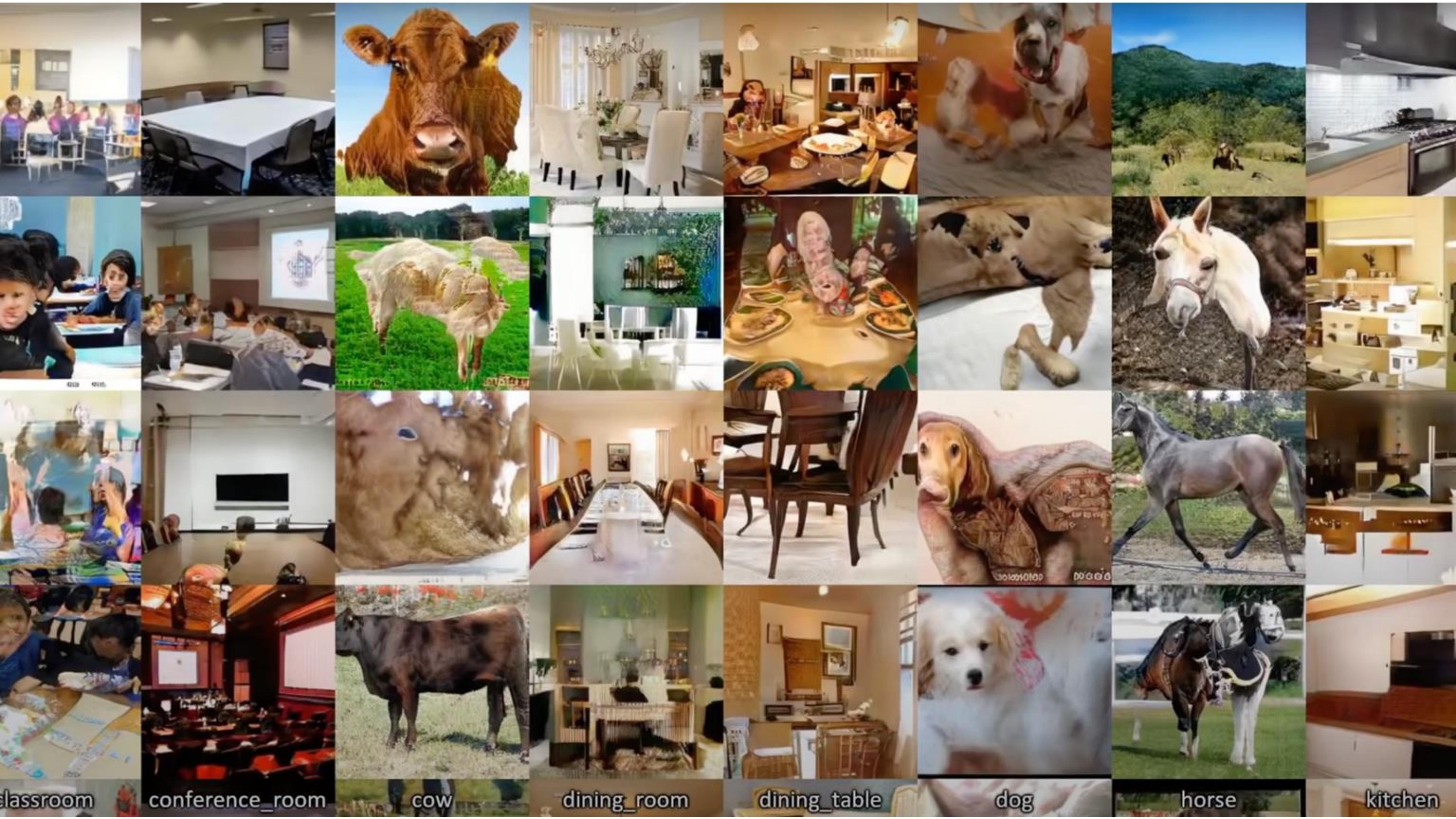
<https://thispersondoesnotexist.com/>

Style GAN [Karras et al., 2018]

Style GAN 2 [Karras et al., 2019]

GANs Difficulties

- Training is really hard
 - Unstable
 - Classification easier than generation
- Good for highly structured images like aligned faces
- Not so great at unstructured images



classroom

conference_room

cow

dining_room

dining_table

dog

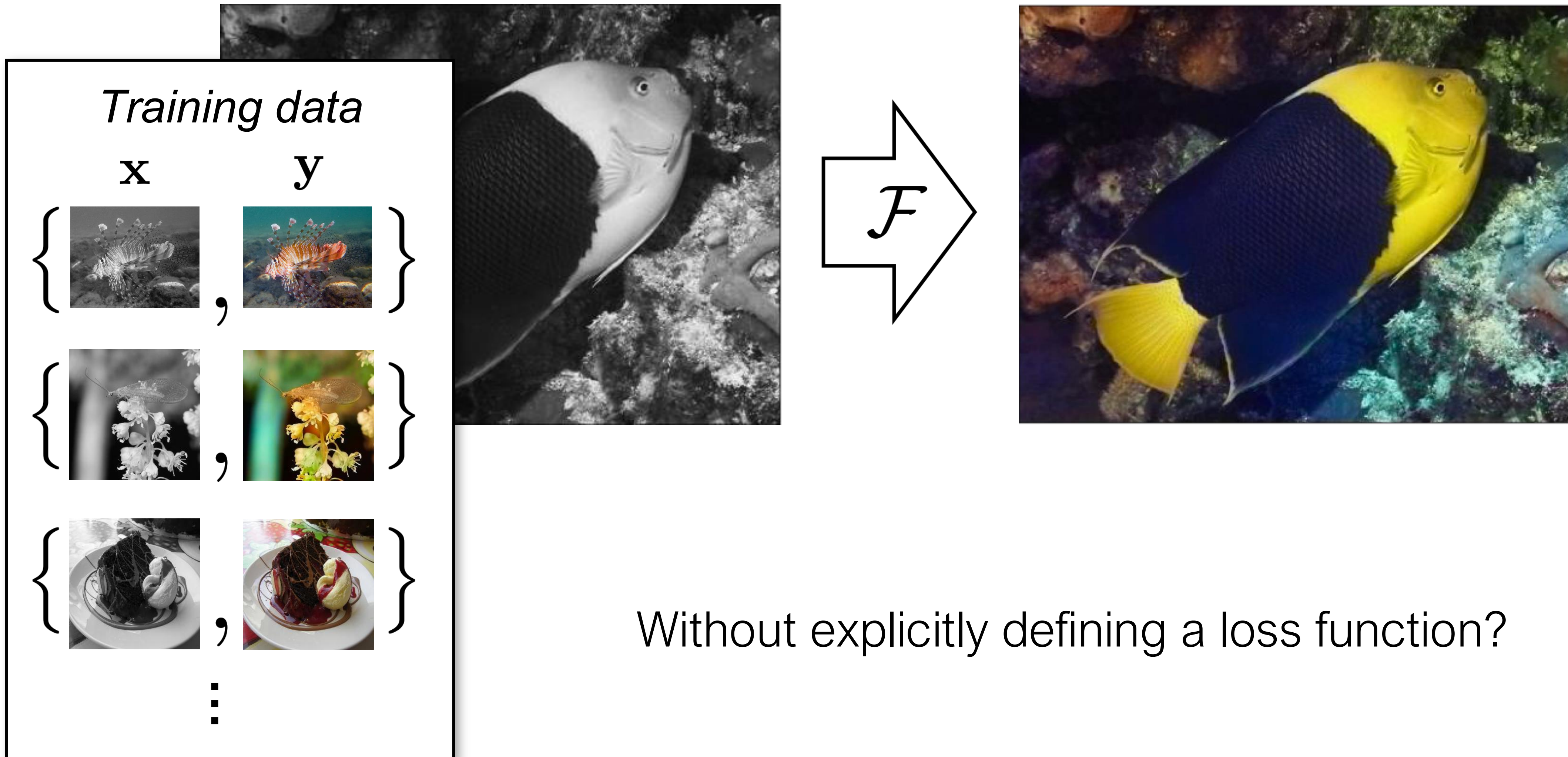
horse

kitchen

Image-to-Image Translation

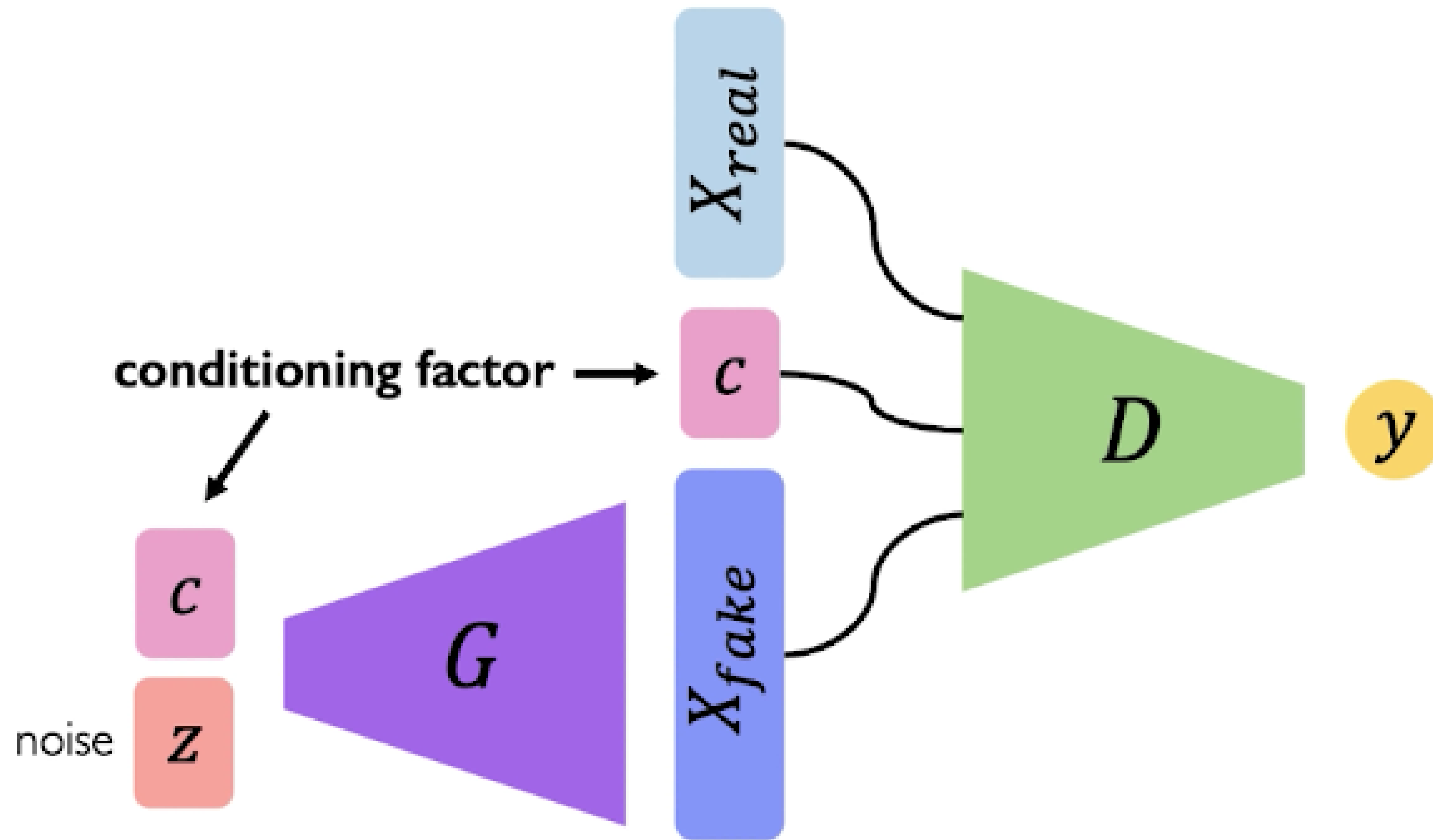
Input $\mathbf{x}$

Output $\mathbf{y}$



Without explicitly defining a loss function?

Conditional GANs (cGAN)



Grayscale → Color

Input

Output

Input

Output

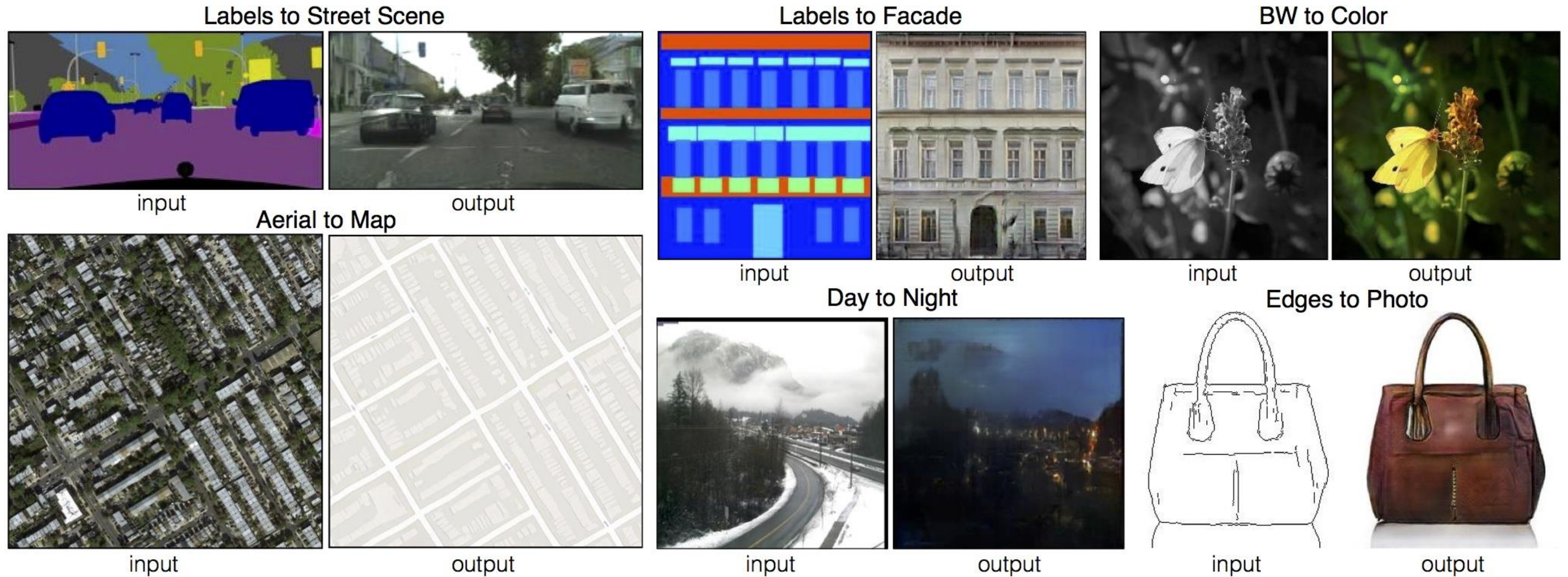
Input

Output



Data from [Russakovsky et al. 2015]

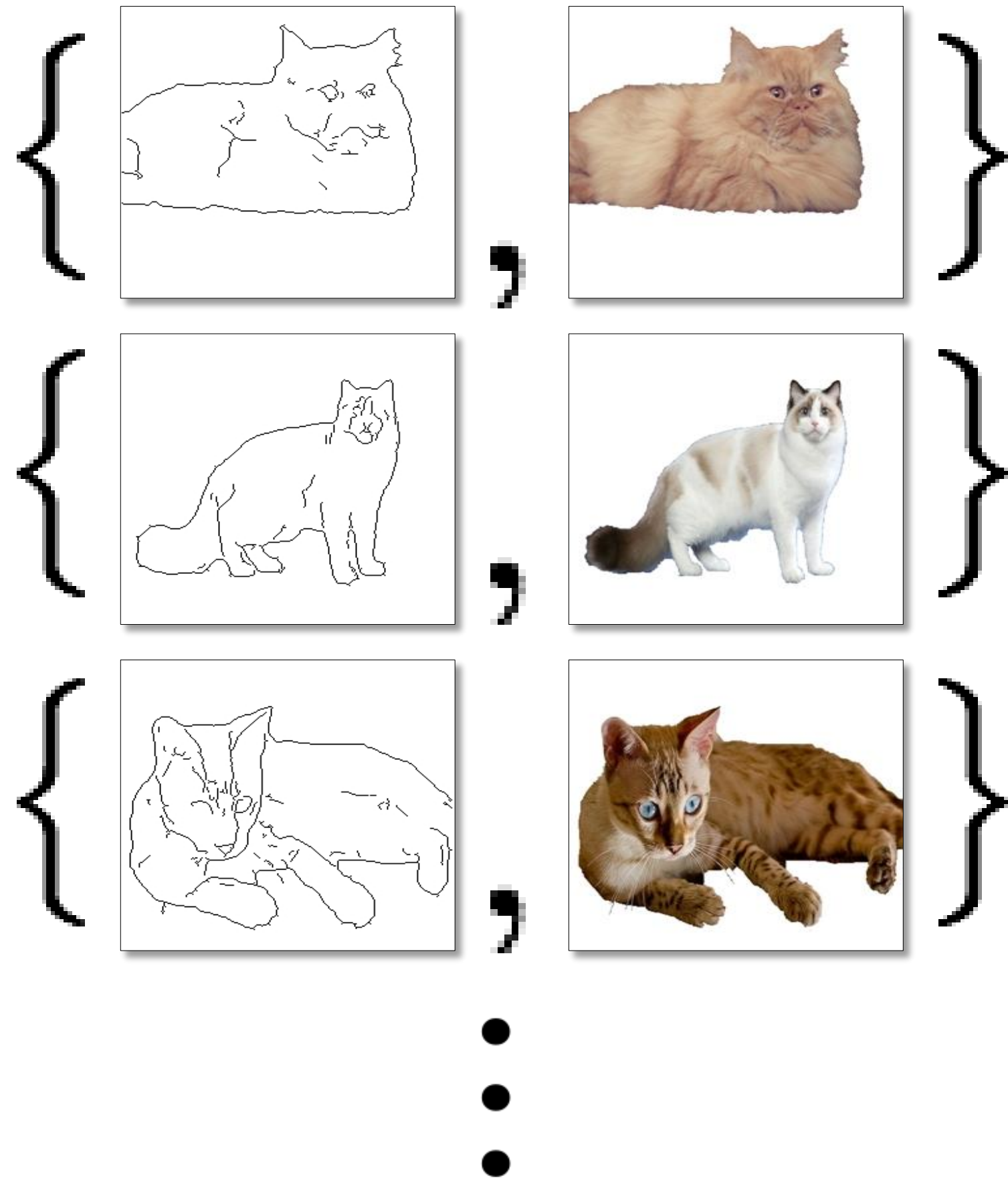
Image-to-Image Translation



Paired data

x_i

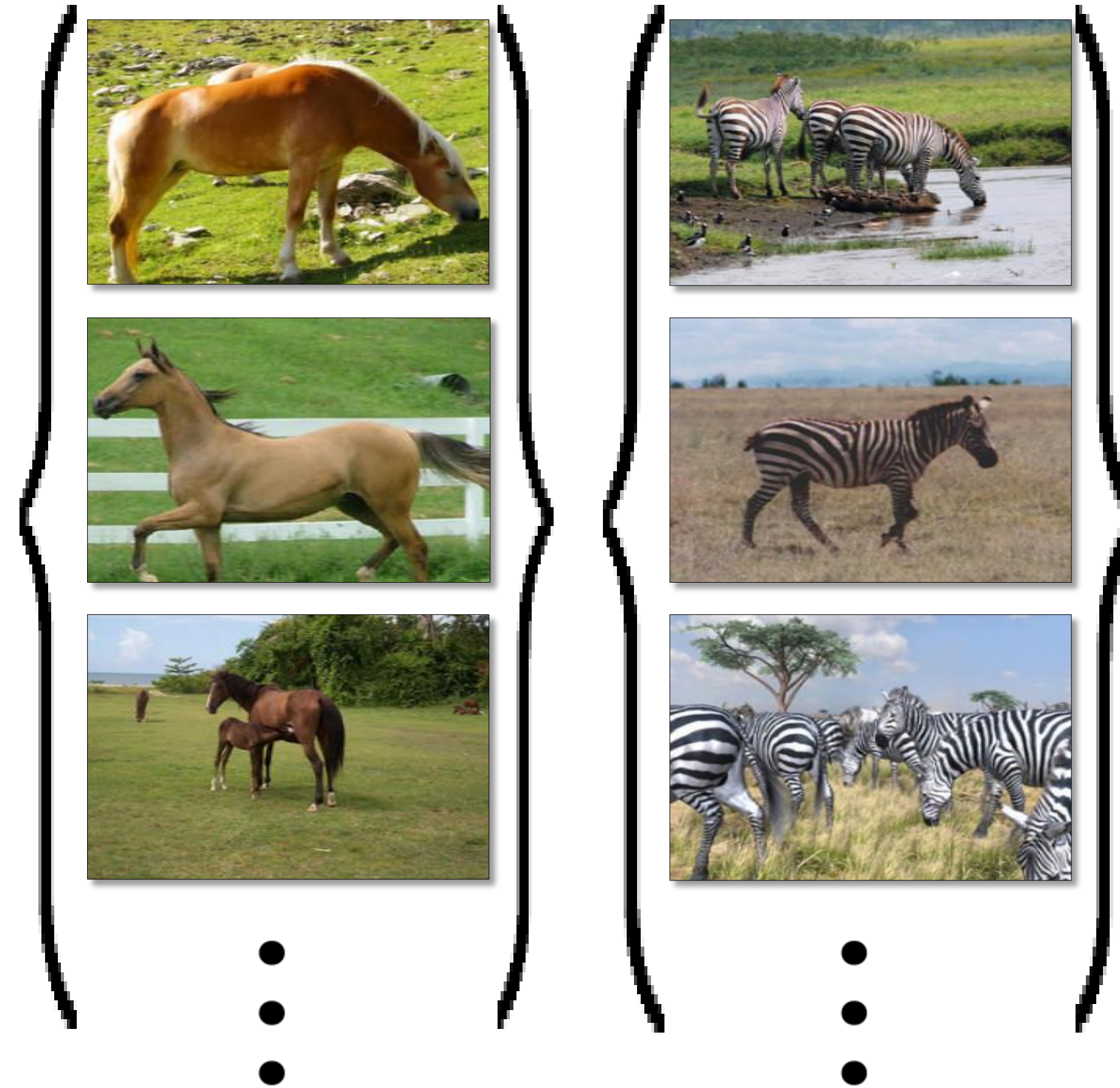
y_i

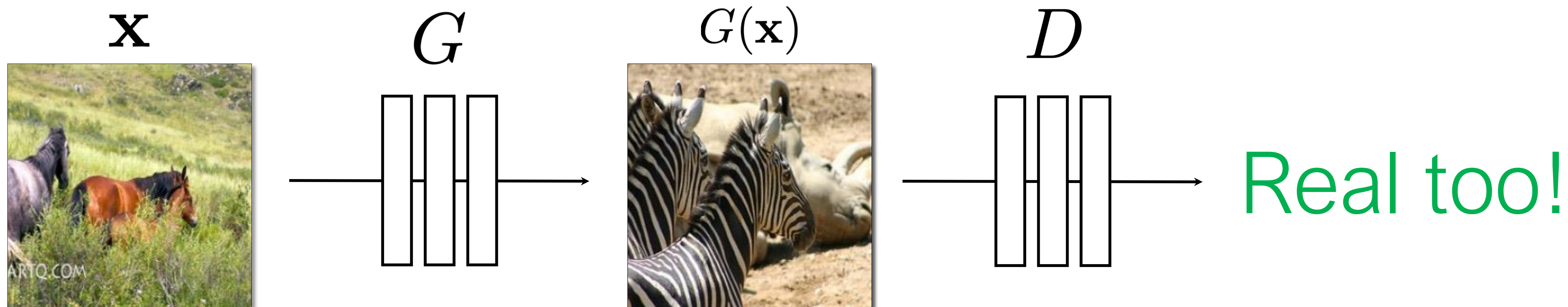
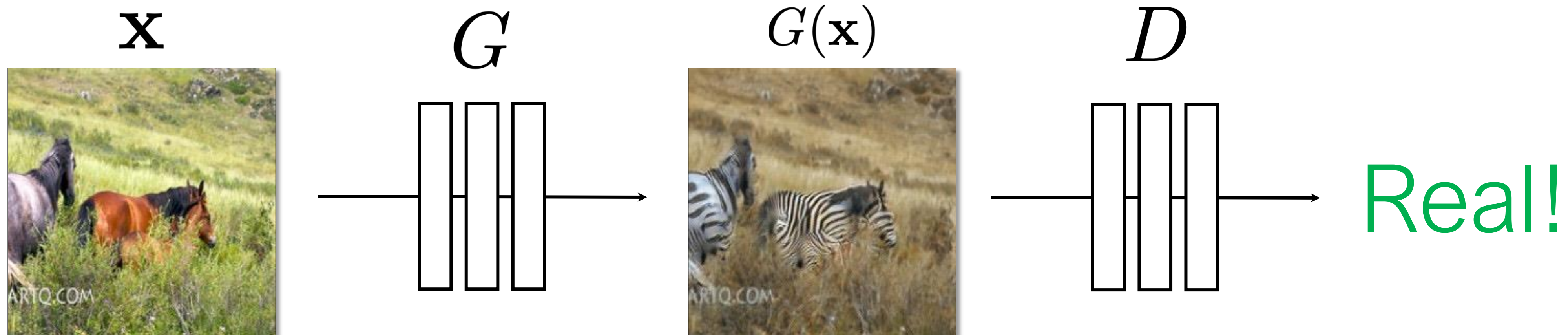


Unpaired data

X

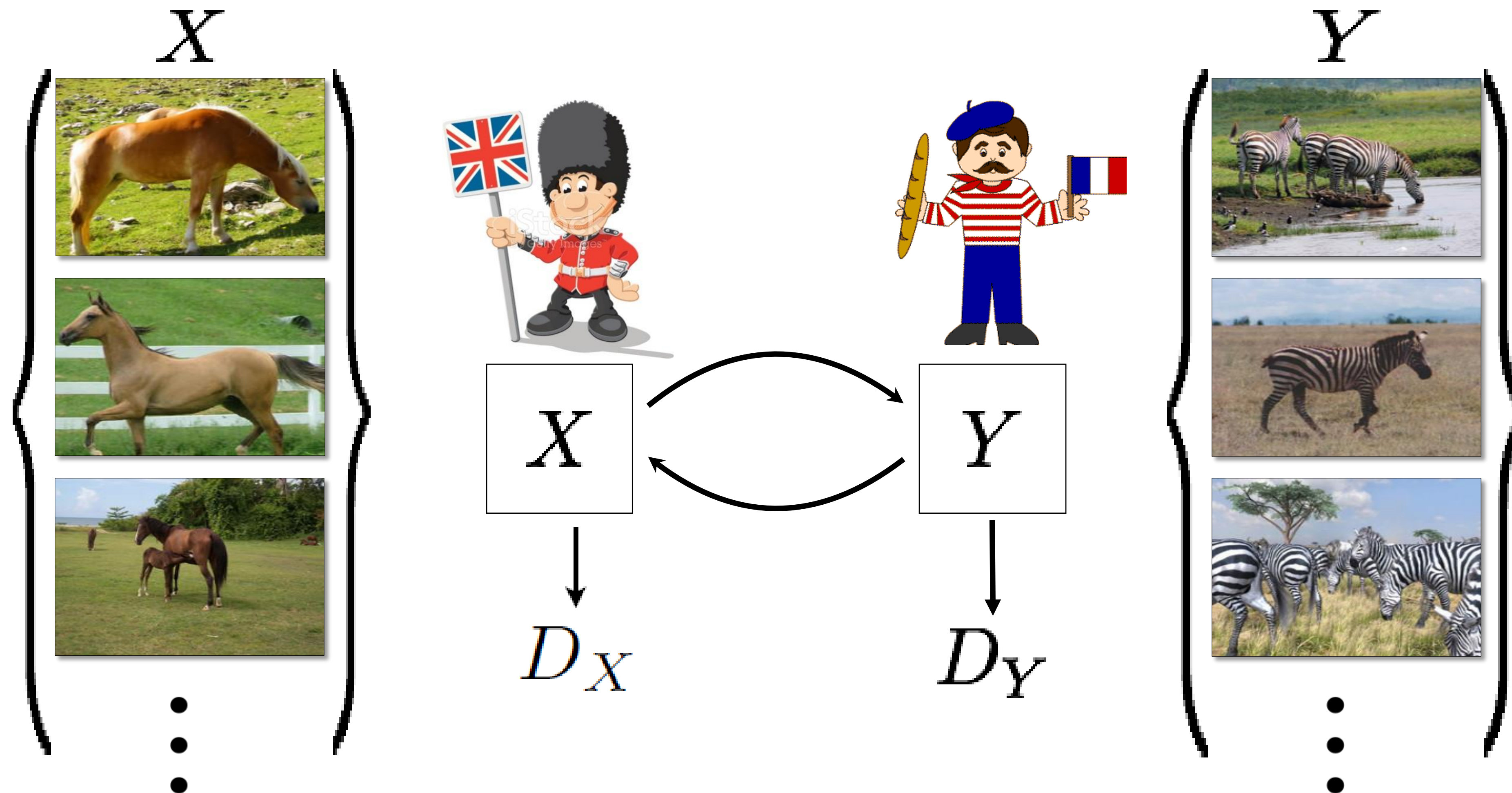
Y



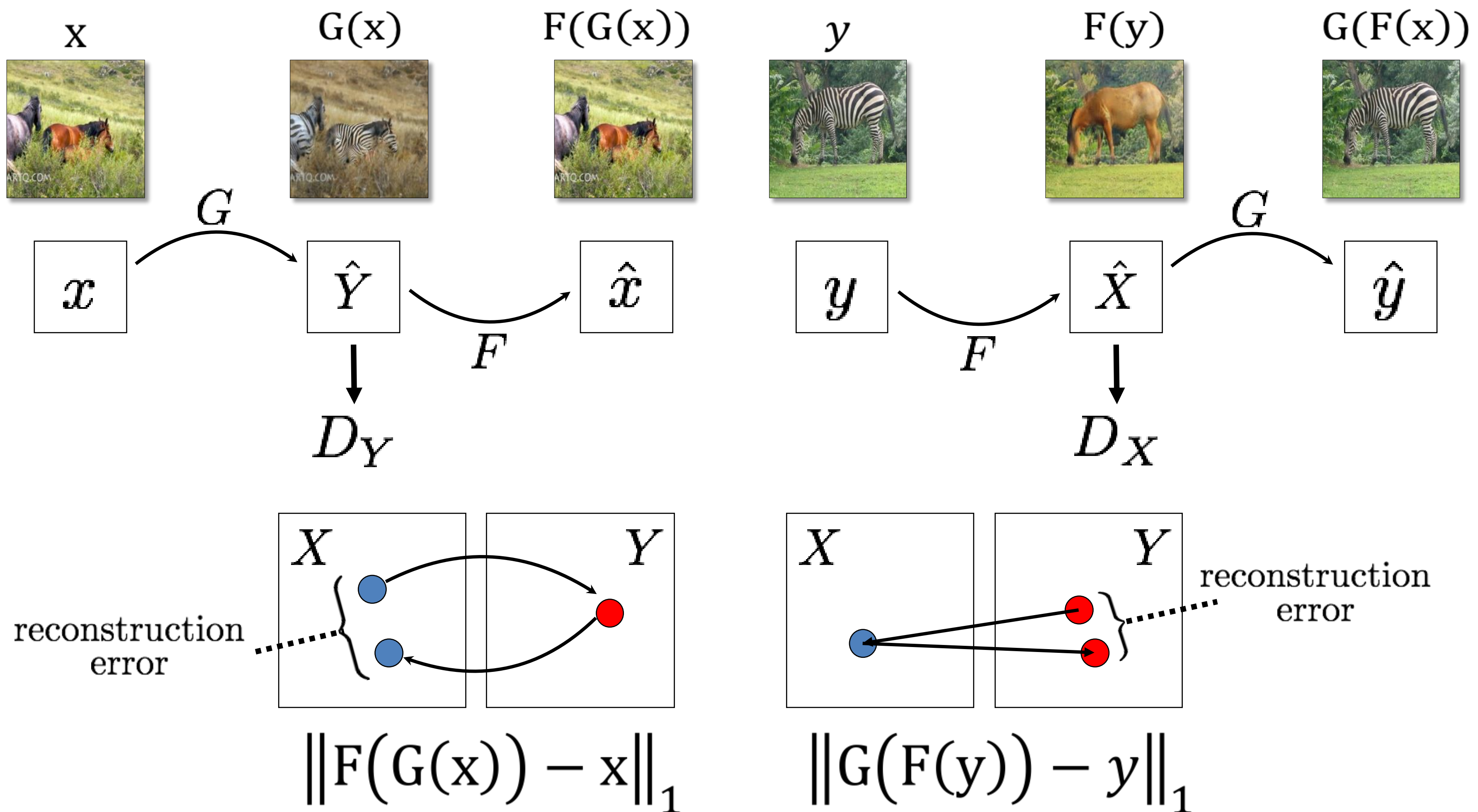


Nothing to force output to correspond to input

Cycle-Consistent Adversarial Networks



Cycle Consistency Loss







Input



Monet



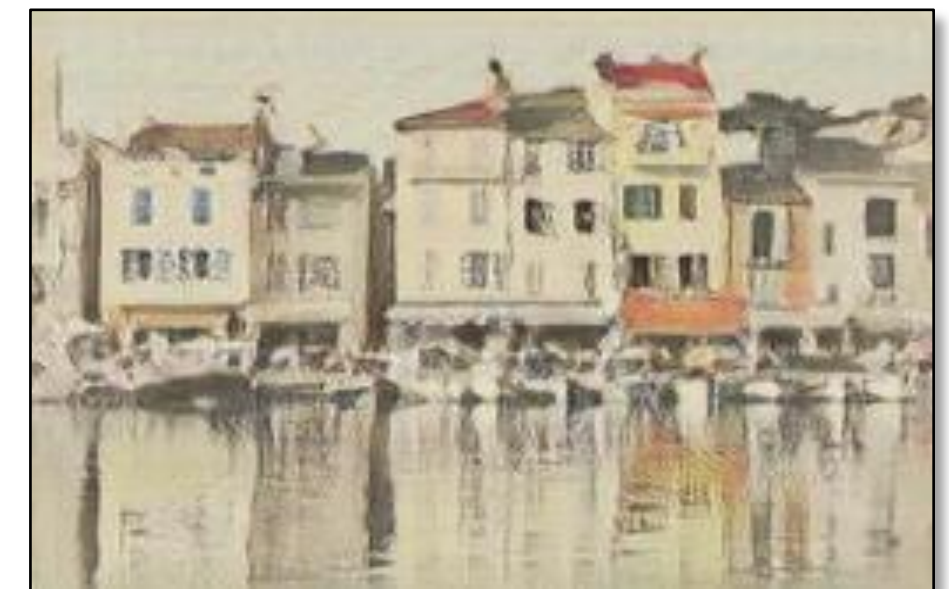
Van Gogh



Cezanne



Ukiyo-e



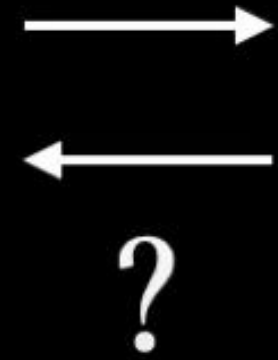
Diffusion Models



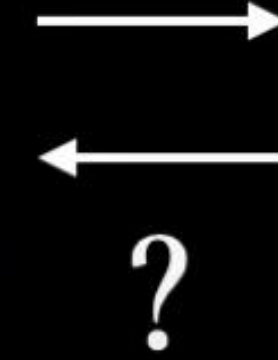
Diffusion Models



x_0



x_t



x_T

Diffusion Models

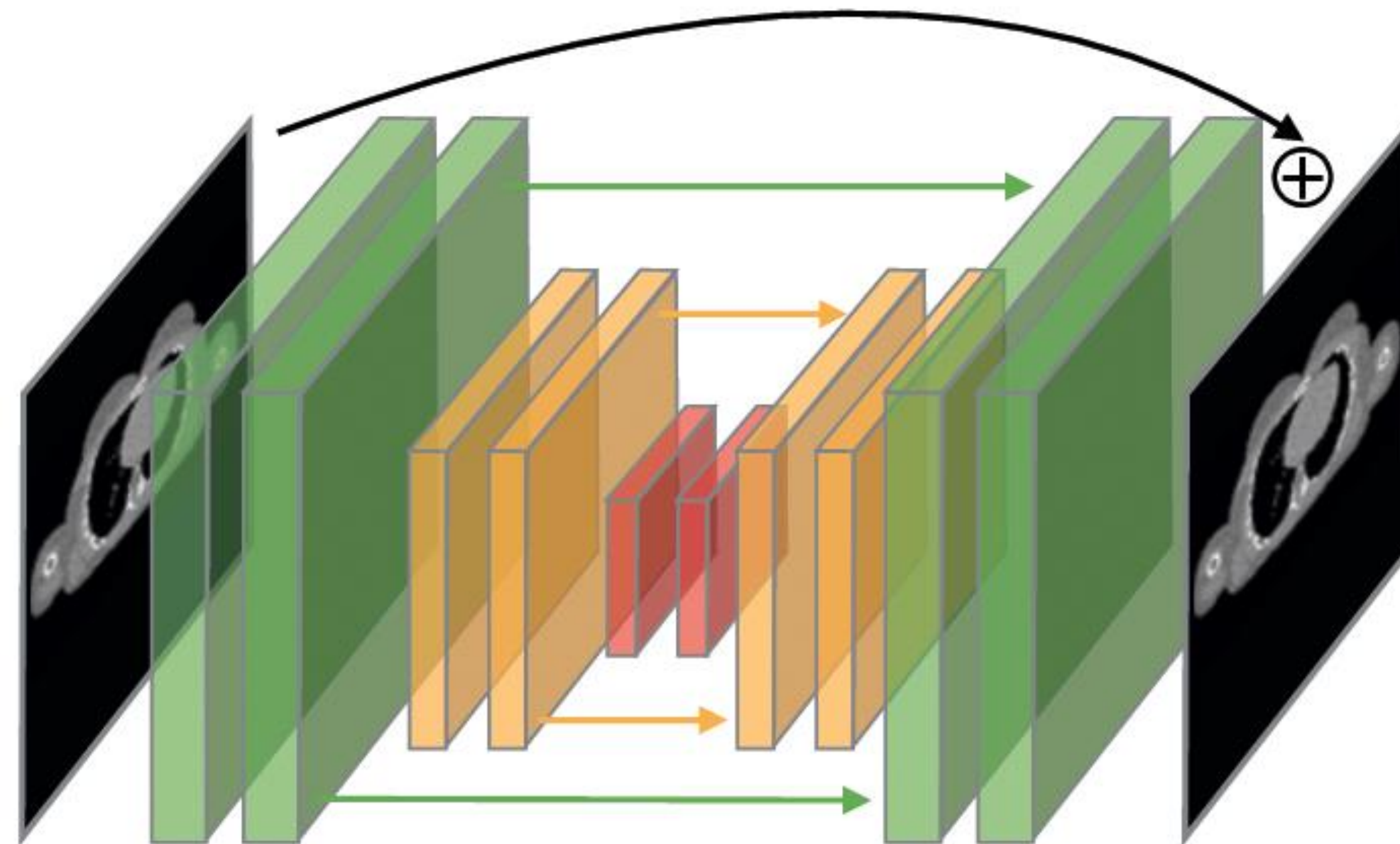
Forward: Add noise to image (easy)



Backward: Train network to denoise

Denoising: neural networks

- Can train neural networks to denoise images
- Typically U-net style architecture
- Network learn to predict inverse of noise

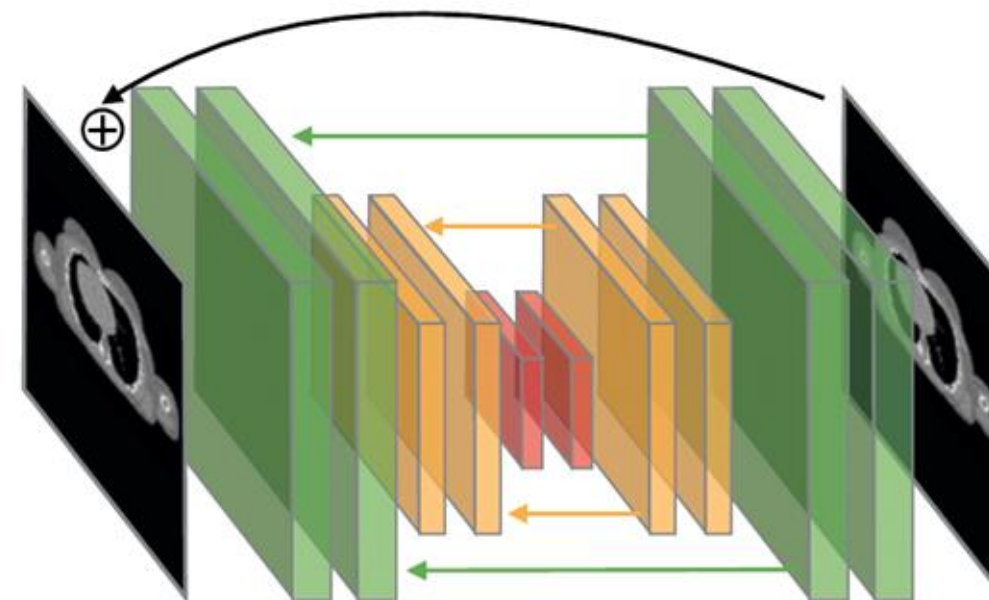


Training denoising

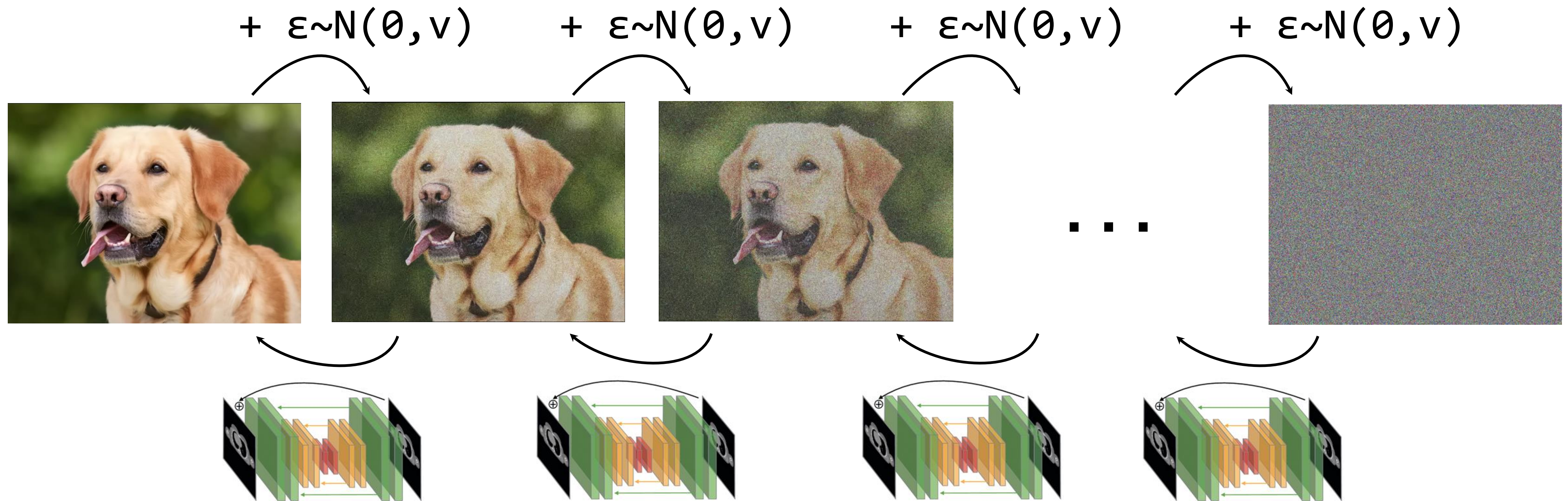
- Easy to generate training data
- Take an image, add noise



$$+ \epsilon \sim N(0, \sigma)$$

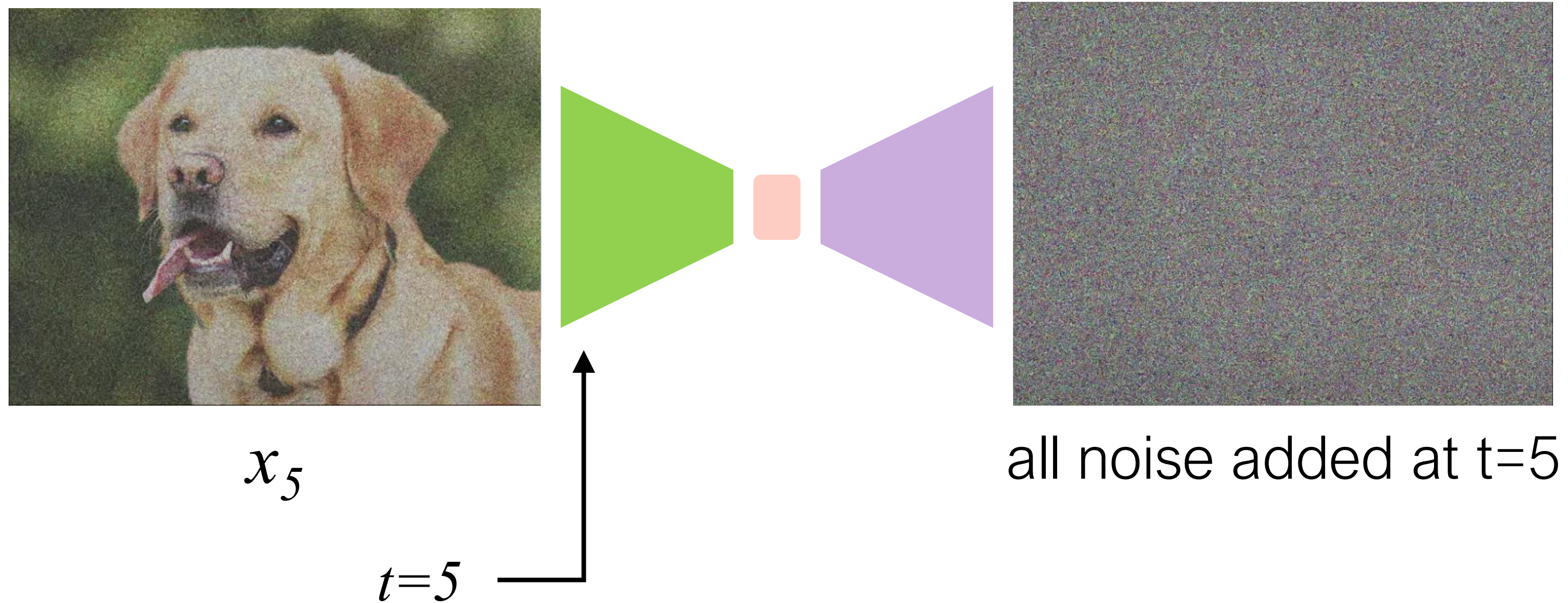


Denoising Diffusion Probabilistic Model (DDPM)



The network predicts ϵ at each step

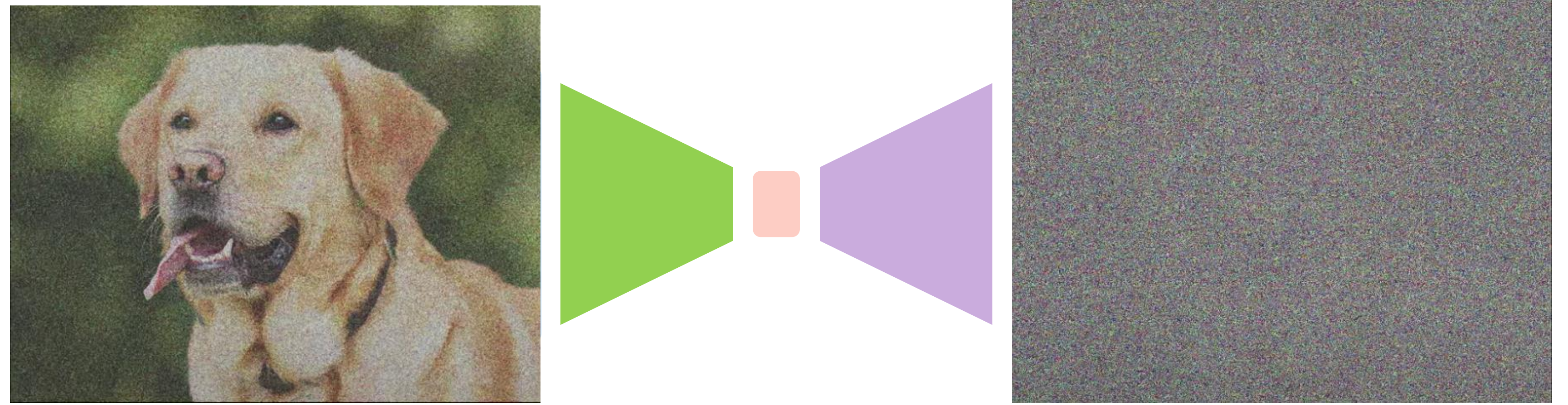
Diffusion Models



During training:

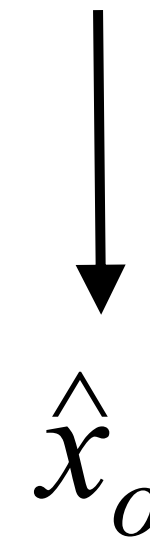
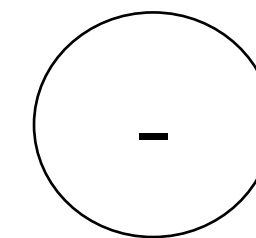
- Pick a random source image
- Pick a random time step
- Add noise based on the noise schedule
- Update the network based on the prediction

Diffusion Models



During inference:

- Start at $t=T$
- Predict the added noise
- Subtract and get an estimate to x_0
- Add noise until $t-1$
- Repeat the process until $t=0$



$\hat{x}_0$

Better samples than GANs

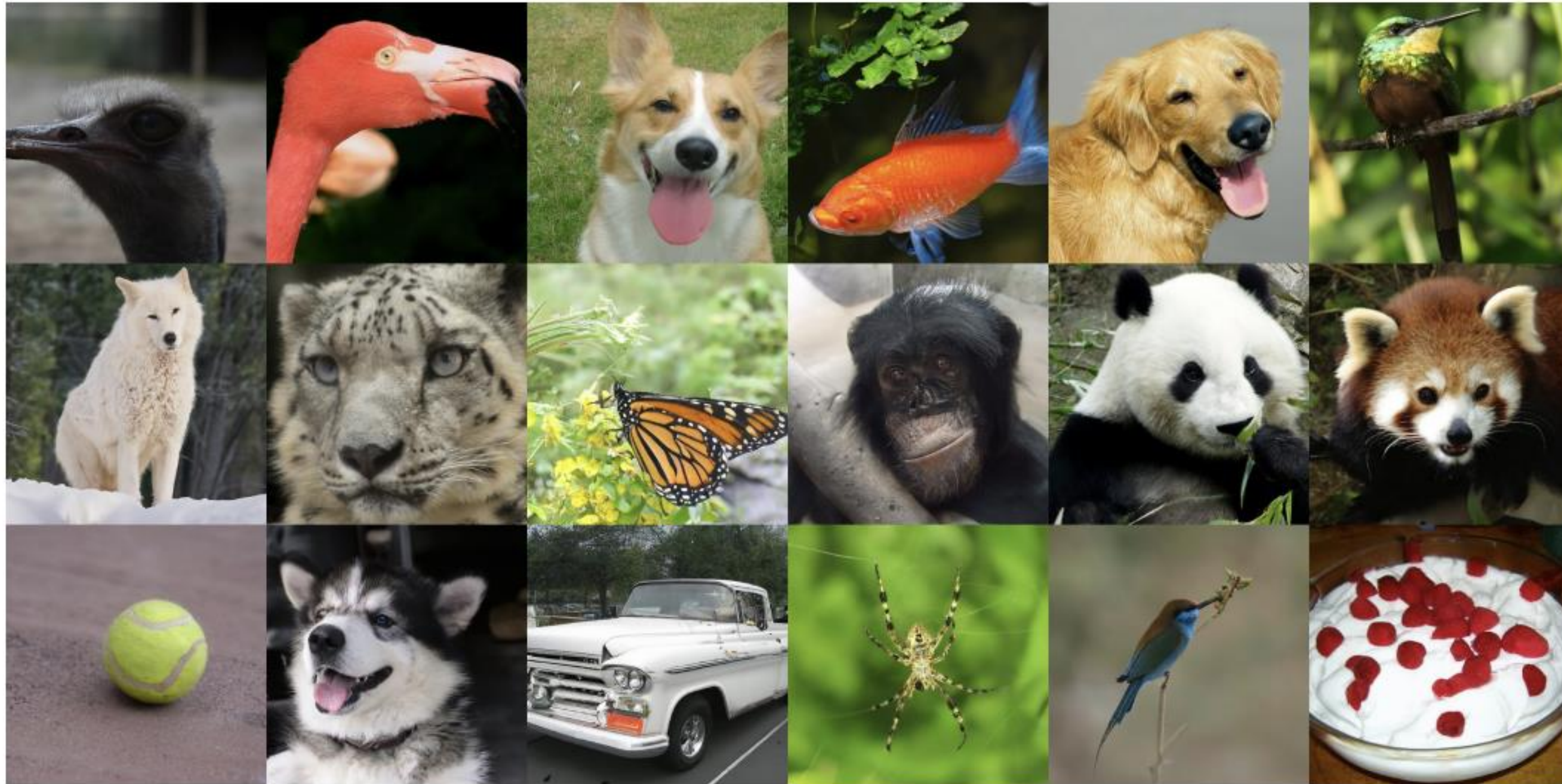
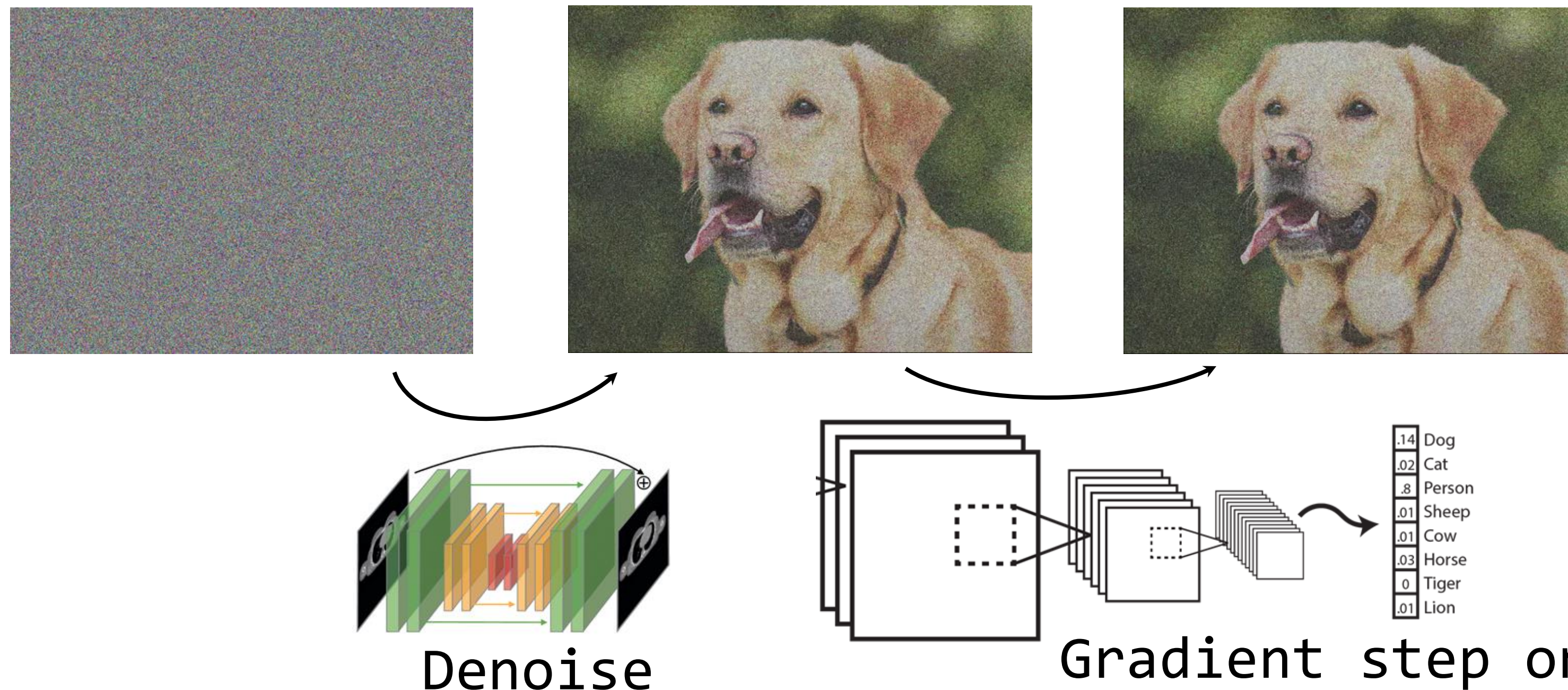


Figure 1: Selected samples from our best ImageNet 512×512 model (FID 3.85)

Conditioning DDPM sampling

- Train classifier on noisy images
- U-Net reverses one step
- Classifier gives loss to adjust noisy image



Classifier-Free Guidance

Train a single model both conditionally and unconditionally; combine the two predictions at inference, with a guidance scale w that trades diversity for prompt fidelity.

Training: randomly drop the condition

~90% of the time

Image x + condition c (e.g. "golden retriever")

Train: $\varepsilon(x, c)$

conditional prediction

~10% of the time

Image x + condition replaced by $\emptyset$ (null)

Train: $\varepsilon(x, \emptyset)$

unconditional prediction

One model that knows both

Inference: combine the two predictions

$$\varepsilon'(x, c) = \varepsilon(x, \emptyset) + w \cdot [\varepsilon(x, c) - \varepsilon(x, \emptyset)]$$

w = guidance scale

Effect of the guidance scale

$w = 1$

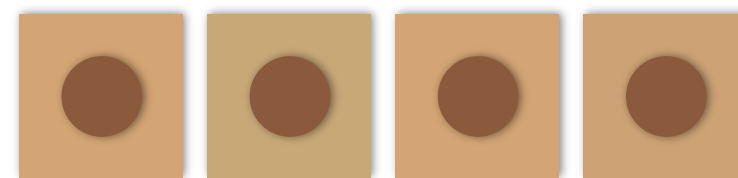
no extra guidance



high diversity, loose to prompt

$w = 3$

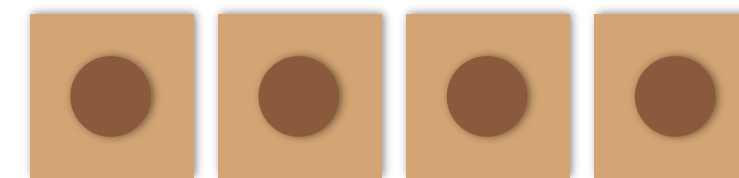
moderate



balanced

$w = 7.5$

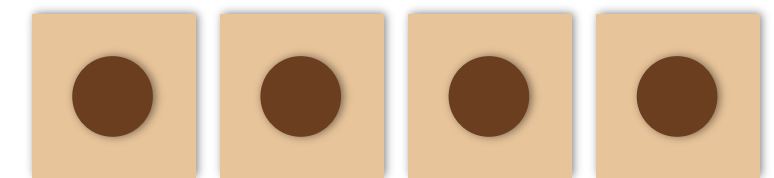
typical default



strong adherence to prompt

$w = 15$

very high



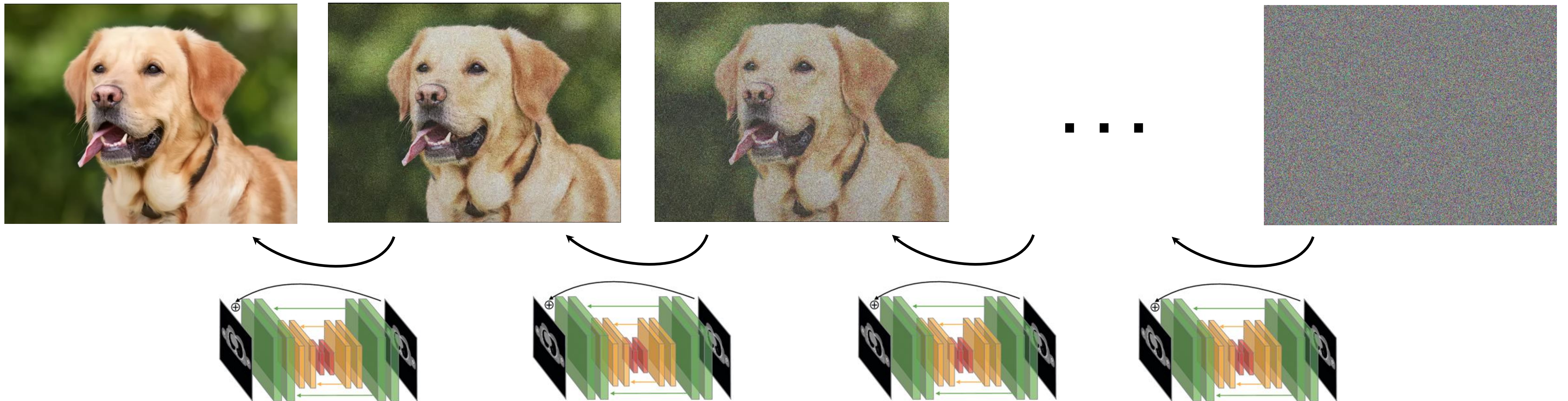
over-saturated, low diversity

more diverse, less faithful

less diverse, more faithful

DDPMs are slow

- Typically need lots of steps (1000-4000)
- Generation needs 1k+ U-net evaluations



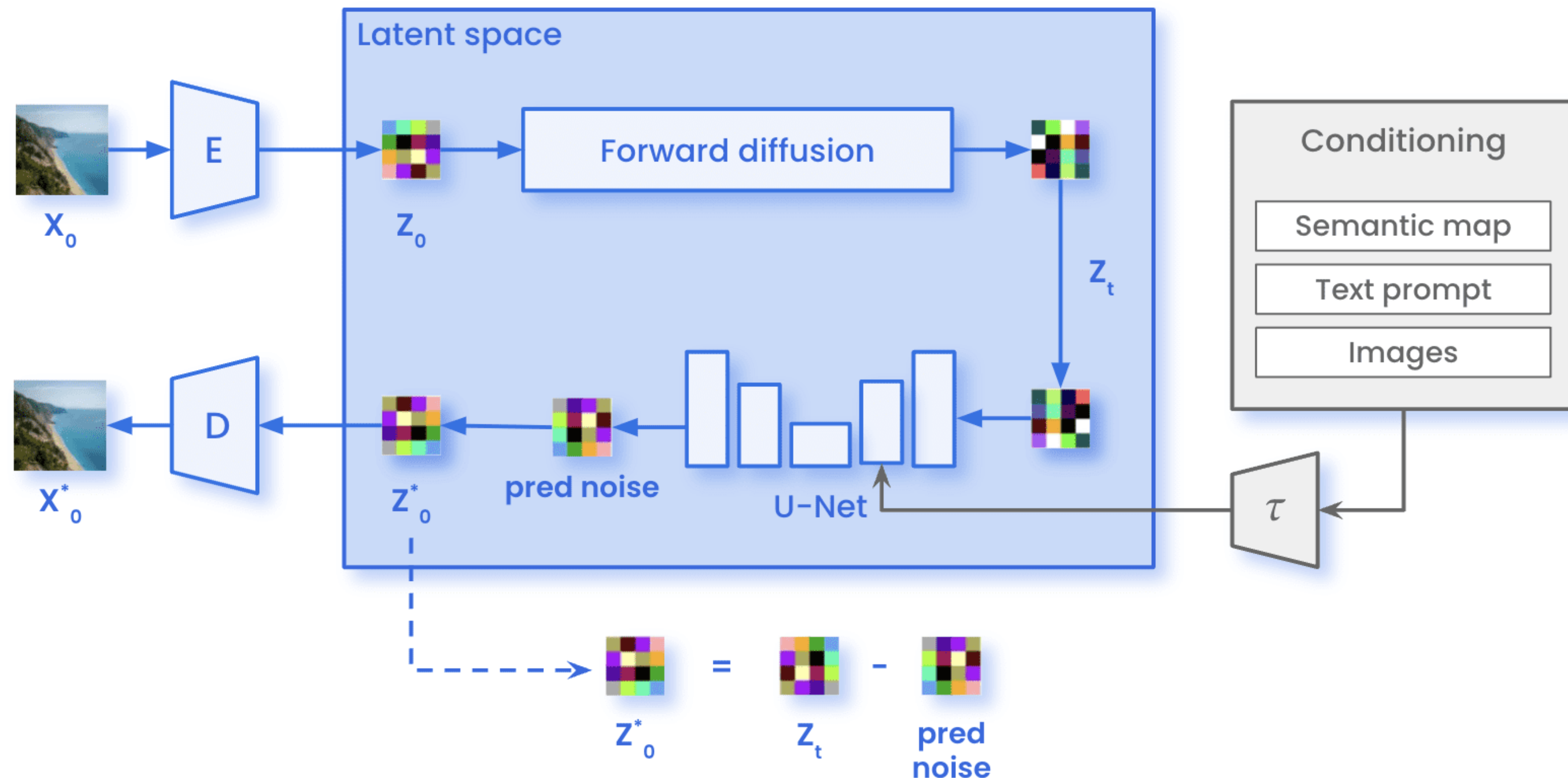
Latent Diffusion Models

(Stable Diffusion)

Why run diffusion in latent space?

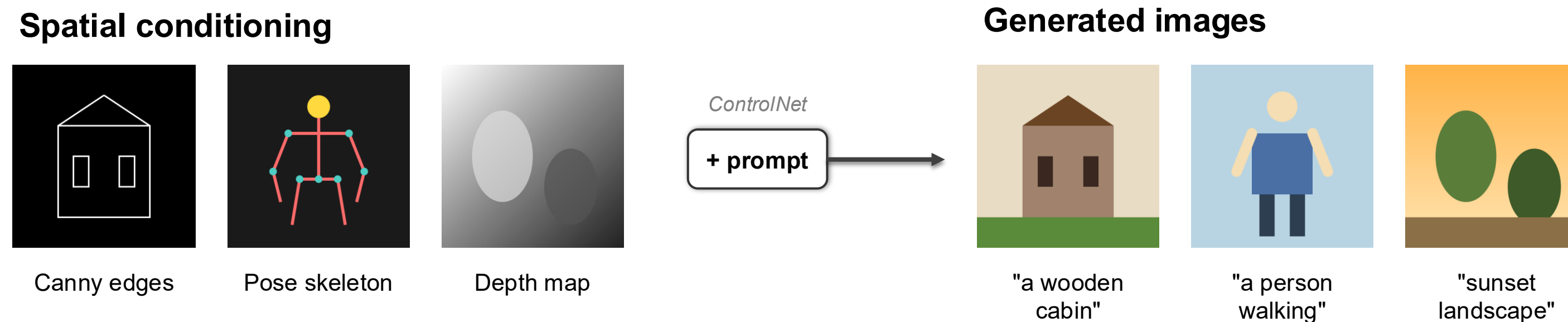
Pixel-space diffusion is expensive (e.g. $512 \times 512 \times 3 = \sim 786\text{K}$ dims).

Compress with a pretrained autoencoder, then diffuse in a small latent.

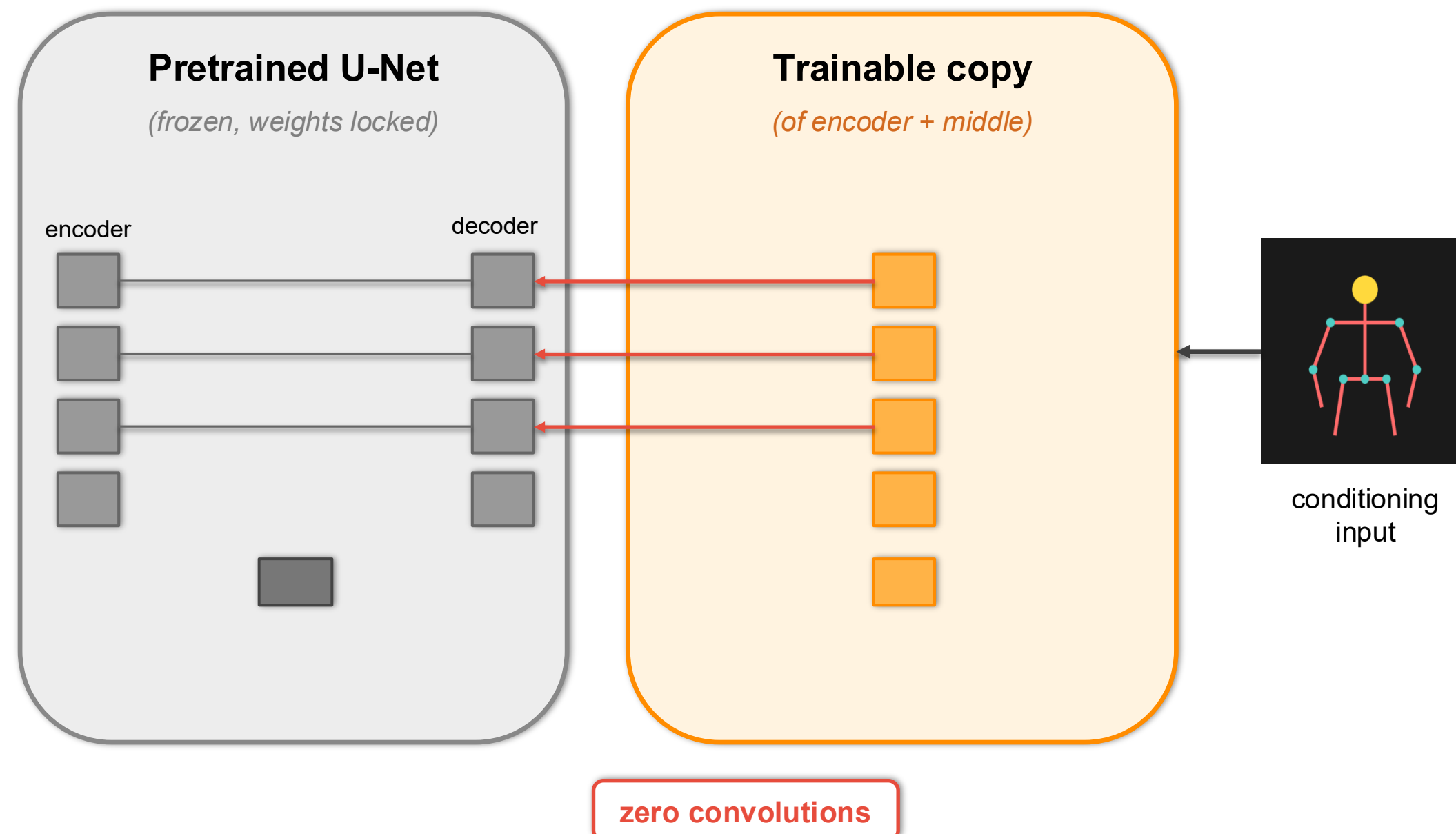


Controllable Generation: ControlNet

Spatial conditioning (edges, pose, depth, ...) added to a frozen Stable Diffusion via a trainable side branch with zero-conv connections.



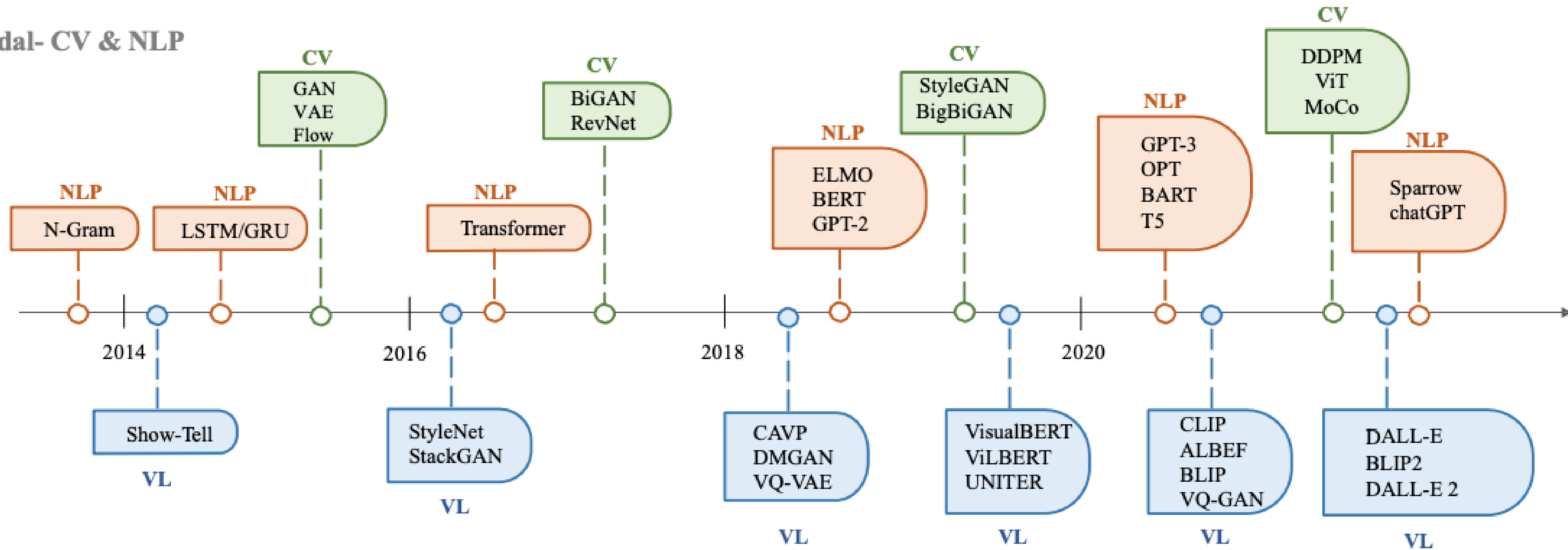
Architecture



Key ideas

- Pretrained weights stay frozen
→ generative knowledge preserved
- Trainable copy learns the new conditioning
- Zero convolutions
→ weights initialized at zero
→ training starts as a no-op, no harm done
- Modality-agnostic
→ same architecture works for any spatial signal

Unimodal- CV & NLP



Multimodal – Vision Language